A Closed-Loop Bilevel Framework with Routing-in-the-Loop Evaluation for Flexible Job Shop Scheduling and Conflict-Free AGV Routing

QI YU, School of Cyber Science and Engineering, Southeast University, PR China

WEI LI*, School of Computer Science and Engineering, Southeast University, PR China

Integrated scheduling of processing and transport in flexible job shops usually treats travel times as a constant matrix indexed by location, i.e. it assumes that vehicles do not interfere with one another. When a finite fleet shares a sparse corridor network, travel time is no longer fixed data but the outcome of routing decisions (in this paper, routing means conflict-free path planning of vehicles on the corridor network); one job's occupancy of a corridor affects the operation times of other jobs. We study the following problem: in shops where transport takes longer than processing and congestion concentrates on a few shared corridors, each operation can be processed by one of several robotic arms that are homogeneous in capability but heterogeneous in efficiency, while AGVs must travel conflict-free on this network. Machine flexibility turns this coupling into an operational trade-off: when the corridor to a fast arm is congested, a slower arm with better connectivity may be the better assignment. Prior work has not measured this trade-off by including it in the evaluation objective of the search.

We propose a *closed-loop bilevel framework*: the upper level decides machine assignment and operation sequencing, and the lower level generates conflict-free paths by time-window reservation; routing information from the lower level enters the decisions in two ways. The first is *routing-in-the-loop evaluation* (the evaluation path): the fitness of each candidate solution is the makespan after conflict-free lower-level routing, so yielding delay enters the optimization objective of the search instead of serving only as a post-hoc check. The second is *reservation-table-aware dispatching* (the decision path): when a vehicle is selected, a trial query of both transport legs is run on the current reservation table, and the vehicle with the earliest actual delivery is chosen, instead of selecting a vehicle by travel times estimated under the assumption of an unimpeded trip. To keep each dispatching evaluation *low-overhead*, we give two speed-ups—admissible lower-bound pruning and winner-path reuse—and prove that they only reduce computation and change neither the selected vehicle nor the final timetable.

Experiments are run on 10 controlled instances (contention intensity from 6.6% to 33.5%), with 10 random seeds per instance, an equal wall-clock budget, and paired tests with multiple-comparison correction. Starting from the open-loop structure in the literature (the search ignores conflicts, and vehicles are dispatched by unimpeded travel time), the complete method reduces the realized makespan by 21.35% (94 wins, 5 losses). Most of the reduction comes from the closed loop: with the dispatching rule fixed and only the search made to count conflict-induced delay, makespan already falls by 18.86%, and the reduction is significant per instance on the vast majority of instances; reservation-table-aware dispatching is a much smaller effect, significant only when all instances are pooled in one test. The two mechanisms each work on their own and do not amplify each other.

Additional Key Words and Phrases: flexible job shop scheduling, conflict-free routing, closed-loop bilevel framework, routing-in-the-loop evaluation, reservation-table-aware vehicle dispatching

*corresponding author

Authors' Contact Information: Qi Yu, 230240012@seu.edu.cn, School of Cyber Science and Engineering, Southeast University, Nanjing, Jiangsu, PR China; Wei Li, xchlw@seu.edu.cn, School of Computer Science and Engineering, Southeast University, Nanjing, Jiangsu, PR China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

ACM Reference Format:
Qi Yu and Wei Li. 2026. A Closed-Loop Bilevel Framework with Routing-in-the-Loop Evaluation for Flexible Job Shop Scheduling and Conflict-Free AGV Routing. 1, 1 (October 2026), 46 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

1.1 Background and motivation

Flexible job shop scheduling with transportation times has long rested on a convenient assumption: the time a job needs to move between two machines is a constant, read from a matrix indexed by location pairs. This is the dominant convention in the literature [22], and recent models still widely adopt it [7, 14, 16]. The assumption is reasonable when the fleet is small enough relative to the corridor network that vehicles rarely meet. Real shops, however, are not built that way: a few vehicles share a sparse corridor network, and a single-lane segment cannot be traversed in both directions at the same time; once fleet density is high enough, the fleet's own traffic turns from a secondary disturbance into the main cause of delay [9]. Once this is acknowledged, travel time is no longer fixed data but the outcome of two decisions: the path a vehicle takes (we call this conflict-free path planning routing), and the order of reservations on contended segments. The duration of a transport therefore has two parts: the travel time computed from distance, and the time spent waiting at berths or detouring because corridors are occupied by other vehicles. The latter means that one job's transport can delay the operations of other jobs.

Such delay, passed from job to job, leaves room for optimization only if the scheduler can adjust the assignment of operations, and machine flexibility is exactly what allows an operation to be reassigned to another arm (i.e. to change its processing location). When an operation may be processed by any robotic arm in an option set, and the processing time depends on the chosen arm, the assignment decision selects both the destination and the processing duration. Suppose an operation may be given either to a fast arm at the far end of the shop or to a slower arm right next to the load/unload point. Under a constant travel-time matrix the comparison is arithmetic: a shorter processing time plus a longer travel time, against a longer processing time plus a shorter travel time. With corridor contention it is different, because the corridor to the fast arm may be exactly the one that every other vehicle is using, and the waiting incurred there is invisible to the constant matrix. A slower arm with better connectivity may then be the better choice; and at different congestion levels of the same instance, the conclusion may differ again. This trade-off requires two ingredients at once: heterogeneous optional arms give the scheduler destinations it can switch between, and distinct paths in the corridor network make vehicles wait and detour differently on the way.

Figure 1 uses a recomputable minimal instance to show that the same pair of assignments favours the fast arm when evaluated with a constant travel-time matrix, and the slow arm when evaluated with conflict-free routing. The instance has 4 jobs and 2 AGVs: one operation of the focal job J1 may use M1 ($t^P = 3$) or M2 ($t^P = 14$); the other three background jobs can be processed only on M3; finished jobs must be returned to the load/unload point v_0. The only route to the fast arm M1 is an exclusive trunk corridor, while the slow arm M2 lies on a separate branch; the background jobs cross the trunk repeatedly and keep it occupied. All times in this instance are in dimensionless time units: below, "taking x" denotes a duration and "$t = x$" an instant measured from the start ($t = 0$).

The numbers in Figure 1 are of two kinds: (a) the bottom gives the round-trip time of the focal job J1 on its own, used to determine the minimum difference in processing time between the fast and the slow arm; (b) the bar heights are the instance makespans when all four jobs are scheduled together, used to compare the assignment rankings produced by the two evaluation methods. The bottom of Figure 1(a) gives J1's unimpeded round-trip time (outbound + processing

+ return): J1 leaves v_0 and is returned to v_0 after processing, with travel times taken from shortest paths and no other vehicles assumed on the way. When J1 is assigned to M1, the path is $v_0 \to v_1 \to v_2 \to$ M1, one way taking $1+\tau+1 = 8$ and the round trip 16; adding processing, which takes 3, gives 19 in total; when J1 is assigned to M2, the path is $v_0 \to v_1 \to$ M2, one way taking $1 + 2 = 3$ and the round trip 6; adding processing, which takes 14, gives 20 in total. Ignoring the other vehicles, the fast arm is better if and only if $19 < 6 + t^P_{\text{slow}}$, i.e. $t^P_{\text{slow}} > 13$; with $\tau = 6$ and $t^P_{\text{fast}} = 3$, this instance takes $t^P_{\text{slow}} = 14$, so the round-trip times of the two assignments differ by only 1 time unit and the fast arm wins by the smallest possible margin. Only because the fast arm is better under unimpeded conditions does the choice of the slow arm under conflict-free routing, shown below, constitute a ranking reversal.

The bar heights in Figure 1(b) are the instance makespan $C_{\max}$, i.e. the time at which the last finished job returns to v_0, obtained by the same decoder as in the rest of the paper. All four jobs start from v_0 and are returned to it; J2, J3, and J4 can be processed only on M3 (outbound and return legs taking 8 each, processing taking 3 each); the two AGVs can operate in parallel.

Under the constant matrix, vehicles do not interfere with one another, and yielding in corridors does not enter the evaluation. If J1 is assigned to M1, then $C_{\max} = 41$: in the first wave, J1→M1 and J2→M3 run in parallel, each with the outbound leg taking 8, processing 3, and the return leg 8, and both are back at v_0 at $t = 19$; in the second wave, J3 and J4 are both transported to M3; they leave together at $t = 19$ and, the outbound leg taking 8, arrive at $t = 27$; J3 is processed from $t = 27$ to $t = 30$ and returns at $t = 38$, while J4 waits until M3 is free at $t = 30$, is processed until $t = 33$, and, the return leg taking 8, is back at $t = 41$, the latest of all. If J1 is assigned to M2, then $C_{\max} = 54$: J1's outbound leg takes 3, processing 14, and the return leg 3; AGV1 waits at M2 until $t = 17$ and is back at v_0 at $t = 20$; J2 still returns at $t = 19$. From then on the operations of the two vehicles are no longer synchronized; J4 arrives at M3 at $t = 43$ and returns to v_0 at $t = 54$. The matrix evaluation still chooses the fast arm.

Now the same pair of assignments is evaluated by conflict-free routing. With J1 assigned to M1, $C_{\max} = 95$: J1's own unimpeded round-trip time is still 19, but its outbound leg occupies the trunk during $t \in [1, 7)$ and its return leg during $t \in [12, 18)$; J2 waits 17 time units at the trunk entrance and arrives at M3 at $t = 25$; J2–J4 then pass through the trunk one after another, and J4 returns to v_0 at $t = 95$. With J1 assigned to M2, $C_{\max} = 60$: J1 takes the branch and does not occupy the trunk, and J2 waits only 1 time unit on v_0–v_1; the background jobs still yield to one another, and J4 returns to v_0 at $t = 60$. The slow arm is now better. The machine assignment and the operation sequence are identical under both evaluations; only the way the makespan is computed differs: the former uses the constant travel-time matrix and ignores yielding, while the latter uses conflict-free routing and counts yielding wait.

1.2 Gap relative to prior work

A setting close to real AGV shops requires three conditions to hold at once: operations have optional machines, with processing times that depend on the machine; vehicles travel on an explicit road network with exclusive corridors, and travel times are determined jointly by the paths of all vehicles rather than taken from a constant matrix; and vehicles do not conflict with one another. Most existing work covers only part of these: flexible job shop scheduling with transportation times represents travel by a constant travel-time matrix, and vehicles do not interfere with one another; conflict-free path planning handles vehicle travel, but the origins, destinations, and release times of tasks are given externally and do not change with the schedule. Few works satisfy all three. Van Os and Basten [21] formalize the problem as flexible job shop scheduling with conflict-free transportation constraints, and solve small benchmark instances to proven optimality with logic-based Benders decomposition. This paper also studies flexible job shop scheduling with conflict-free routing, but focuses on a class of settings that prior work has not taken as a dedicated object

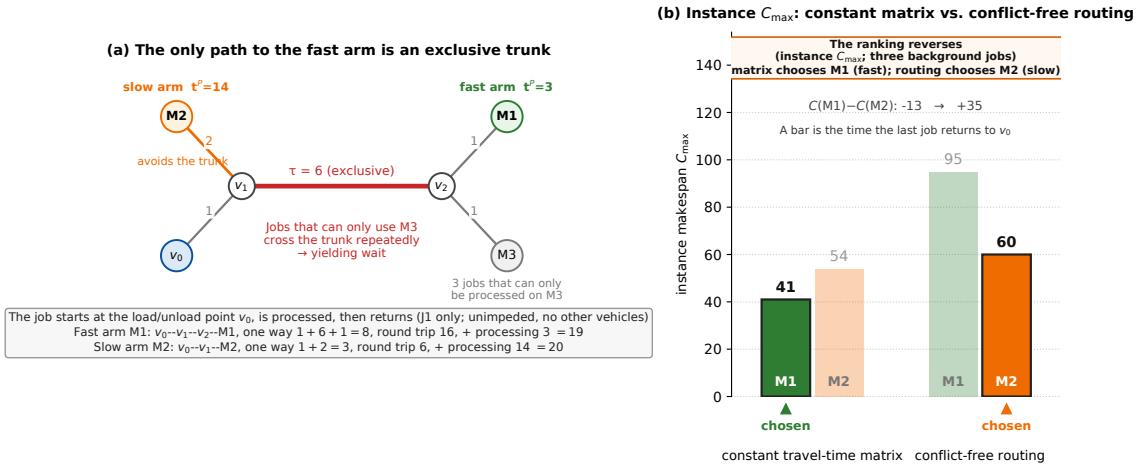


Fig. 1. Motivating instance: the same assignment decision is ranked in opposite order by two evaluations. (a) The only route to the fast arm M1 is an exclusive trunk ($\tau = 6$); the slow arm M2 lies on a short corridor that does not pass through the trunk; three jobs that can be processed only on M3 cross the trunk repeatedly. The bottom gives the unimpeded round-trip times of the focal job J1 (M1: 19; M2: 20), used to calibrate the minimum heterogeneity required for a reversal. (b) Bar height is the instance makespan $C_{\max}$ (4 jobs, 2 AGVs, same decoder as in the main text). Under the constant matrix, $C_{\max}$ is 41 versus 54 and the fast arm is chosen; under conflict-free routing, the same pair of assignments gives 95 versus 60 and the slow arm is chosen, so the difference changes from −13 to +35. All times are in dimensionless time units. This figure is an existence example; of the 1200 parameter combinations around this instance, 205 (17%) show the reversal.

of study: transport takes longer than processing ($\bar{T}_t/\bar{T}_p \approx 4.0$, whereas common public instances reach at most 0.12), robotic arms are connected by single-lane corridors, each corridor is exclusive over its whole length, vehicles may wait only at berths, and congestion concentrates on a few shared corridors. In such shops, vehicle contention becomes the main constraint, and the quality of the same assignment changes with the level of congestion; we study this trade-off with contention intensity as a controlled variable, and target sizes that exact decomposition has difficulty solving. At these sizes, existing heuristics either schedule first and resolve conflicts afterwards (Sun et al. [19]), or feed back the transport durations realized by the path layer once without modifying the assignment (Li et al. [10]), or embed routing in decoding without measuring its benefit and cost (Lyu et al. [13]). Therefore, in this setting, it has not yet been measured under equal compute how the assignment should trade off between "fast but congested" and "slow but clear" arms, how much gain is obtained by counting yielding delay in the evaluation of the search, and at what level of contention this gain is worth its computational cost.

1.3 Approach

To address this gap, we adopt a bilevel structure: the upper level decides machine assignment and operation sequencing, and the lower level generates conflict-free paths by time-window reservation; routing information from the lower level enters the schedule in two ways, through evaluation and through decisions (detailed in the next two paragraphs), and we measure the contribution of each to makespan separately.

The *evaluation path* takes the fitness of each candidate solution as the makespan after the lower level has completed conflict-free routing, so yielding wait enters the objective that the search optimizes, rather than serving as a post-hoc feasibility check. One conflict-free decode costs more than an order of magnitude more computation than a decode

based on unimpeded shortest-path times that ignore contention; under fixed compute, a higher cost per evaluation reduces the number of evaluations that can be completed. Lyu et al. [13] already embed time-window routing in the decoding of a genetic algorithm, and Li et al. [10] list "extending this into a fully iterative loop" as future work, but neither reports the computational cost of this practice or its gain relative to open-loop scheduling. We measure this trade-off under an equal wall-clock budget.

The *decision path* uses *reservation-table-aware dispatching*. A transport consists of two legs: empty pickup and loaded delivery. For each candidate vehicle, the lower-level router runs a trial query on the current reservation table for the earliest feasible times of the two legs in turn (the trial query does not write to the final timetable), and the vehicle with the earliest feasible delivery is selected, rather than the vehicle whose arrival is earliest by an estimate based on unimpeded shortest-path times. This rule replaces the commonly used idle-first dispatching (a typical single-attribute rule in multi-AGV dispatching [12], also used by Li et al. [10]). To keep each evaluation *low-overhead* under an equal wall-clock budget, we give two speed-ups: admissible lower-bound pruning and winner-path reuse. Both skip only routing trial queries that cannot affect the result, and we prove that the selected vehicle and the timetable are exactly the same as without the speed-ups.

1.4 Main results

To measure the contributions of closed-loop evaluation and of reservation-table-aware dispatching separately, we construct a four-level baseline ladder whose first two levels represent two common process structures in the literature: **B0** is the open-loop structure, which first schedules by unimpeded travel times and then hands the schedule to conflict-free routing for execution (methods that schedule on a constant matrix [7, 14, 16] take exactly this structure when executed on a real road network, and the framework of Li et al. [10] also belongs to this class); **B0+** is the two-stage structure, which freezes the schedule and selects vehicles by the reservation table only in the second stage (e.g. Sun et al. [19]); **B1** is closed-loop planning with rule-based dispatching; **B2** is our method. The four configurations form a 2×2 design along two factors: whether the search counts conflict-induced delay, and whether vehicles are selected by unimpeded time or by trial queries on the reservation table. Only a comparison that changes a single factor can attribute a gain to the corresponding mechanism. The four configurations share the same upper-level search, the same router, and the same wall-clock budget, and differ only in the two places above; the differences therefore reflect only the structure, not how strong the implementation of each published algorithm is. The makespan that an open-loop plan reports for itself is often infeasible on the actual corridors, so all schedules are executed by the same conflict-free router, and all comparisons use the realized makespan after execution.

The results show significant gains along both factors. With the dispatching rule fixed, including yielding wait in the fitness reduces makespan by 18.86% ($p < 10^{-4}$); this is the contribution of closed-loop evaluation; within the closed loop, replacing rule-based dispatching with reservation-table-aware dispatching reduces makespan by 2.45% ($p = 0.0054$) relative to closed-loop planning with rule-based dispatching; this is the contribution of the dispatching mechanism; from the open-loop structure to the complete method, the end-to-end reduction is 21.35% (94 wins, 5 losses, $p < 10^{-4}$). The three comparisons use different baselines and cannot be added directly. The two low-overhead speed-ups make each dispatching evaluation 2.33 times faster and cut router calls by 47.4%, lowering the cost multiple from 12.2 to 5.23; this is the precondition for dispatching to yield a net gain under an equal wall-clock budget.

Both mechanisms are significant, but they are *not substitutes for each other*: reservation-table-aware dispatching under the open loop yields 4.59% ($p < 10^{-4}$), far less than the closed-loop 18.86%; the end-to-end 21.35% is slightly below the value 22.6% predicted by independent composition of the two, which shows that the two mechanisms each

work on their own and do not amplify each other. If only a single contrast from the open loop to the complete method were reported, the few percentage points from dispatching would be merged with the closed-loop gain into one term; we therefore report the two factors separately.

Results grouped by layout, vehicle-to-arm ratio, and flexibility are given in Section 5.4. Per-instance tests are Holm-corrected by family. After correction, the closed-loop gain remains significant on 9 of the 10 instances and rises roughly with contention intensity, so it can be interpreted along the layout axis; reservation-table-aware dispatching is significant on none of the instances (2 before correction), and across the four levels of the vehicle-to-arm ratio the sign of its gain varies and none reaches significance. Dispatching is therefore significant only when all instances are pooled in one test; with 10 seeds per instance, the sample is not sufficient to identify on which class of instances it is stably positive.

These conclusions rest on two comparison rules (detailed in Section 5.1.2). First, the computational cost per evaluation can differ by about two orders of magnitude across methods; comparing at an equal number of generations would let the more expensive method use more compute, so we align wall-clock time. Second, each contrast changes only one factor (whether the search counts delay, or how vehicles are selected); a contrast that changes both at once yields only a combined effect and cannot tell whether the gain comes from the closed loop or from dispatching.

1.5 Contributions

(1) **Instances: instance families with controllable contention intensity, for studying the reassignment trade-off between fast and slow arms.** When machine flexibility, heterogeneous processing efficiency, and corridor contention are all present, and the schedule targets the makespan with yielding wait counted, a slower arm with better connectivity may be the better assignment if the corridor to a fast arm is congested. Models that schedule on a constant travel-time matrix cannot reflect this comparison, because the matrix contains no yielding wait [19]; models with conflict-free routing but a fixed machine for each operation do not involve this comparison either, because there is no reassignment [10]. Van Os and Basten [21] and Lyu et al. [13] also consider flexible assignment under conflict-free routing, but do not study its effect with contention intensity as a controlled variable; targeting shops where transport takes longer than processing and congestion concentrates on shared corridors, we construct instance families with controllable contention intensity, to examine how high contention must be before counting yielding wait in the schedule brings a gain.

(2) **Framework: a closed-loop bilevel framework, with a measurement of the contribution of routing-in-the-loop evaluation to makespan.** *Routing-in-the-loop evaluation* takes the fitness of each candidate solution as the makespan after the lower level has completed conflict-free routing, so yielding wait enters the optimization objective of the search. We measure the gain and the computational cost of this practice relative to the open loop (the search ignores conflicts) under an equal wall-clock budget, and prove that every chromosome can be decoded into a conflict-free, deadlock-free timetable, so the search needs no repair operator.

(3) **Mechanism: reservation-table-aware dispatching, and two speed-ups that keep it *low-overhead*.** When a vehicle is selected, a trial query of both transport legs is run on the current reservation table, and the vehicle with the earliest feasible delivery is chosen. We give two speed-ups, admissible lower-bound pruning and winner-path reuse, and prove that they change neither the selected vehicle nor the final timetable; it is precisely this reduction in overhead that lets the mechanism achieve a net gain under an equal wall-clock budget.

(4) **Evidence: the two paths contribute at different magnitudes, each works on its own, and they are not substitutes for each other.** The closed loop is the larger effect, and nearly all of its per-instance tests are significant; reservation-table-aware dispatching is a much smaller effect, significant only when all instances are pooled in one test; the two do not amplify each other, and changing only vehicle selection under the open loop is far from a substitute for counting yielding wait in the search. Two further inter-layer interfaces (an extra cost on occupied corridor–time slots, hereafter surcharge, and reassignment or staggering guided by yield records) only add overhead without reducing makespan under equal compute; the failure of the surcharge indicates that, before extending the inter-layer interface, one should first check whether the objective of the receiving level already contains the quantity that the interface is meant to pass.

The notation of the paper is summarized in Table 2.

2 Related work

Related work falls into three groups: flexible job shop scheduling with transportation, conflict-free routing and dispatching with exogenous tasks, and integrated work that brings the two into one problem. Table 1 summarizes how each group compares with this paper.

2.1 Flexible job shop scheduling with transportation

Flexible job shop scheduling with transportation (FJSP_PT) brings material movement into the schedule. Xin et al. [22] review progress in the field through 2023 and point out that the dominant modelling convention is a constant travel-time matrix indexed by location pairs: a vehicle needs a fixed time $t(a, b)$ to travel from a to b, which does not change with whether other vehicles occupy the same corridor. For modelling and algorithms of the general FJSP, see Dauzère-Pérès et al. [1]; the public benchmarks of this family are collected by Homayouni and Fontes [6].

Recent work keeps this matrix and advances mainly on solution methods. Meng et al. [14] propose a constraint-programming model for the multi-AGV case and a dual-population genetic algorithm assisted by constraint programming, closing 29 open benchmark instances and improving 27 best-known solutions; Qin et al. [16] combine a quality–diversity archive with Q-learning and design a local search jointly guided by AGVs and the critical path; Jafarian-Moghaddam et al. [7] jointly optimize scheduling and internal logistics in a U-shaped flexible flow shop with a mathematical model and a metaheuristic framework. In all of these works, however, transport durations are taken from the matrix, so vehicles never delay one another through corridor contention; and although critical-chain reasoning is a mature device in job-shop local search [15, 20], a critical path computed on the matrix can only be traced back to a predecessor operation or a busy machine, not to a specific corridor. Xin et al. [22] also note that such assumptions "reduce the complexity of the problem and even change its nature", and that existing instances "differ mainly in size and lack a dedicated design for the values of key problem parameters"; they list the ratio of mean transport time to mean processing time, $\bar{T}_t/\bar{T}_p$, as a quantity that is highly likely to affect algorithm performance.

2.2 Conflict-free routing with exogenous tasks and congestion-aware dispatching

Multi-agent path finding (MAPF) studies the collision-free movement of several agents on a shared graph, and solving it optimally is NP-hard [4]; Kasaura et al. [8] extend Safe Interval Path Planning to two-dimensional road networks and continuous time. For settings with continuously arriving tasks, Zhang et al. [24] optimize offline a road network with edge weights that steers agents away from bottlenecks, raising the throughput of several solvers. Kim and Kim [9]

propose a congestion function, a congestion penalty, and congestion-aware dispatching rules for large AGV fleets. Surveys of multi-AGV management and of mobile-robot control [3, 12] treat dispatching and path planning as two adjacent but separate layers of the control stack, and list idle-first and nearest-vehicle as common single-attribute dispatching rules. In this line of work, however, the release time, origin, and destination of a task are all inputs to the router, which cannot revise the machine assignment that generates these tasks; congestion is also mostly quantified by indirect measures such as local vehicle density or offline edge weights, which can only show that a corridor carries relatively many vehicles, not which transport caused yielding, when, and where.

2.3 Integrating scheduling and conflict-free routing

Work that considers flexible assignment and conflict-free routing together is scarce. van Os and Basten [21] formalize FJSP with conflict-free transportation constraints and propose a logic-based Benders decomposition: a constraint-programming master problem gives the assignment and sequence, and an integer-programming subproblem plans conflict-free paths and returns conflicts to the master as cuts. Most benchmark instances are solved to proven optimality, and about half improve makespan by at least 10% relative to prior heuristics. The exact decomposition, however, applies only to small instances; the authors also list making fuller use of "which conflicts caused the infeasibility" as a direction for future work. Sun et al. [19] propose a two-stage method: a hybrid genetic algorithm first fixes the assignment and sequence, and time-window Dijkstra then resolves conflicts, with every node and every segment holding at most one vehicle at a time; but even when the second stage detects corridor congestion, it cannot change the choices made in the first. Li et al. [10] forbid both node and edge conflicts on a grid and add one feedback step that writes the transport durations realized by the path layer back onto the time axis; but the feedback only shifts start times without revising the assignment, acts on the current solution only, and their benchmark has a single eligible machine per operation. Lyu et al. [13] embed time-window Dijkstra in genetic-algorithm decoding, so that the makespan of every chromosome is computed after routing, with vehicles encoded in the chromosome and conflicts resolved by local rules; but the research focus of that paper is determining the fleet size, its instances are small, it does not compare against open-loop scheduling, and it does not report the computational cost of the embedded routing.

In summary, the gap in existing work lies not in whether vehicle conflicts are handled, but in how routing results enter the schedule, and in what doing so is worth. This paper studies shops in which transport takes longer than processing and congestion concentrates on a few shared corridors (corridors rather than nodes are reserved; see Section 3.5): at sizes beyond the practical reach of the exact decomposition, every candidate solution is evaluated by its makespan after conflict-free routing, and reservation-table-aware dispatching selects vehicles by trial query; the contributions of the two paths are measured separately under an equal wall-clock budget, and instance families with controllable contention intensity are used to examine how the gain varies with contention—two instances with the same $\bar{T}_t/\bar{T}_p$ may differ greatly in how much of their congestion can be bypassed by reassignment, and hence in the room left for scheduling to optimize.

3 Problem statement and model

3.1 Problem statement

An instance consists of n jobs, N_M robotic arms, and N_A vehicles (AGVs), operating on a shop-floor road network represented as a graph. Job j has a fixed chain of n_j operations $O_{j1} \to \cdots \to O_{jn_j}$; operation O_{ji} may be processed on any robotic arm in its option set $\Omega_{ji} \subseteq M$, taking time t^P_{jim} on arm m, a value that depends on the chosen arm. Robotic

Table 1. Comparison of related work with this paper. "Machine heterogeneity" means that an operation has several optional machines and its processing time varies with the machine; "corridor exclusivity" means whether a corridor holds at most one vehicle at any time: on a single-lane corridor, two vehicles following each other in the same direction or meeting head-on may collide or deadlock, so the corridor must be occupied in turn as an exclusive resource, a modelling choice closer to real AGV shops; "transport times used to evaluate schedules" are the transport times the upper level uses when comparing candidate schedules: an "unimpeded estimate" is the shortest-path time computed while ignoring other vehicles, and "routed (incl. yielding)" is the time after conflict-free routing, including yielding wait; "closed-loop gain and cost measured" means whether the gain and the computational cost of putting routing into the search evaluation are measured against the open loop under equal compute.

Work class	Ref.	Machine heterogeneity	Corridor exclusivity	Transport times used to evaluate schedules	Closed-loop gain and cost measured	Research focus
FJSP_PT metaheuristics	[7, 14, 16]	yes	no	unimpeded estimate (constant matrix)	no	solution quality under a constant matrix
Lifelong MAPF	[4, 24]	—	yes	no scheduling (tasks exogenous)	no	network throughput under continuously arriving tasks
Congestion-aware dispatching	[9]	—	no	no scheduling (tasks exogenous)	no	congestion relief for large fleets
Two-stage FJSP + routing	[19]	yes	yes	unimpeded estimate; routed after the schedule is frozen	no	schedule first, then resolve conflicts
RL scheduling + PBS	[10]	no	yes	unimpeded estimate; one post hoc correction	no	hierarchical framework of RL scheduling and multi-vehicle paths
GA with embedded time-window routing	[13]	yes	yes	routed (incl. yielding) for every candidate	no	optimal fleet size
Exact decomposition (LBBD; costly, small instances only)	[21]	yes	no	relaxation bound + conflict cuts	no	proven optimality at small scale
This paper		yes	yes	**routed (incl. yielding) for every candidate**; dispatching by trial query on the reservation table	**yes (equal wall clock)**	**reassignment trade-off between fast and slow arms under corridor contention; gain and cost of the closed loop at equal computing budget**

arms occupy fixed nodes of the graph, and jobs are moved between these nodes by the fleet. Solving an instance means determining four groups of decisions at once:

(i) assign each operation O_{ji} to some robotic arm $m \in \Omega_{ji}$;
(ii) sequence the operations assigned to each robotic arm;
(iii) assign the induced transport tasks to vehicles and sequence the tasks of each vehicle;
(iv) plan a path on the graph for every vehicle move, conflict-free in time and space, with waiting allowed.

The objective is to minimize the makespan $C_{\max}$.

3.2 Modelling assumptions

A1. Static instance and complete information. All n jobs are already at v_0 at time $t = 0$; there are no release dates or due dates. Processing data, the layout G, and the vehicle states are all known at planning time.

A2. Jobs. Each job has a fixed precedence chain. Processing and transport are non-preemptive, jobs cannot be split, and there are no exogenous precedence constraints among different jobs.

Table 2. Notation.

Symbol	Meaning
$J = \{1, \dots, n\}$, O_{ji}	set of jobs; the i-th operation of job j, $i \le n_j$
$M = \{1, \dots, N_M\}$, $\Omega_{ji} \subseteq M$	set of robotic arms; option set of O_{ji}
t^P_{jim}	processing time of O_{ji} on arm $m \in \Omega_{ji}$
$K = \{1, \dots, N_A\}$	homogeneous unit-load vehicles
$G = (V, E)$, τ_e	shop-floor graph; traversal time of corridor $e \in E$
v_0, $\mathrm{loc}(m)$	load/unload point; node occupied by arm m
$t^*(a, b)$	shortest-path time from a to b in G, ignoring contention
$\delta_{\mathrm{ret}} \in \{0, 1\}$	whether a finished job returns to v_0 (baseline value 1)
S_{ji}, C_{ji}, A_{ji}	start, completion, and material-arrival times of O_{ji}
$C_{\max}$	makespan

A3. Robotic arms: partial flexibility and heterogeneous processing times. Each operation O_{ji} may be processed on any $m \in \Omega_{ji}$; most operations satisfy $|\Omega_{ji}| \ge 2$, and full flexibility $\Omega_{ji} = M$ is a special case. Processing times vary with the chosen arm, and the arms need not share a uniform ranking by speed: when $m \ne m'$, t^P_{jim} and $t^P_{jim'}$ need not be proportional. An arm processes only one operation at a time and does not fail during the planning horizon; sequence-dependent setup times are folded into t^P_{jim}.

A4. Buffers and loading. The input and output buffers of each arm, and the storage at v_0, have ample capacity, so blocking does not occur. Loading and unloading times are negligible and do not occupy the arm.

A5. Vehicles. The N_A vehicles are homogeneous and unit-load, and travel at constant speed whether empty or loaded; acceleration, deceleration, and battery charge are not modelled. All vehicles stand empty at v_0 at $t = 0$ and are immediately available. A transport task is a one-way move—from v_0 to the first arm, between two successive distinct arms, or, when $\delta_{\mathrm{ret}} = 1$, from the last arm back to v_0; any vehicle may execute any task, so successive moves of the same job may be relayed across different vehicles. After unloading, a vehicle is not forced back to v_0; it repositions according to its own task chain, and its departure time may be postponed by the routing layer.

A6. Road network and conflict-freedom. G is strongly connected, and the traversal time τ_e of each corridor $e \in E$ is a known constant. Corridors are *exclusive* resources: the two directions of each corridor e share a single reservation, so at most one vehicle occupies e at any time, which rules out both head-on conflicts and same-direction overtaking. Occupancy intervals are half-open, $[t, t + \tau_e)$, so a second vehicle may enter exactly at $t + \tau_e$. Nodes have berths of ample capacity, independent of the traversal resources; waits therefore occur at berths and are always permitted.

A7. Out of scope. Machine and vehicle failures, finite buffers and blocking, battery charging, and dynamic job arrivals [5, 25] are left to future work (item (5) of Section 6.3).

Of these, A1, A2, A4, and A5 are standard assumptions in the FJSP_PT literature [22]. The features that define our problem are A3 (machine-dependent processing times on partially flexible robotic arms) and A6 (contention stated as an explicit exclusive-resource constraint rather than a constant travel-time matrix); as a result, transport duration is an output of the routing decision, not a fixed input parameter. Deadlock-freedom is not an assumption: Proposition 2 in Section 4.3 uses A4 and A6 to prove that any assignment and operation sequence can be decoded into a conflict-free, deadlock-free timetable, so the search need not repair infeasible solutions.

3.3 Decision variables and induced transport tasks

Let $x_{jim} \in \{0,1\}$ indicate that O_{ji} is assigned to robotic arm m; write $\mathrm{m}(j,i)$ for the chosen arm and $d_{ji} = \mathrm{loc}(\mathrm{m}(j,i))$ for the node of that arm. When $\delta_{\mathrm{ret}} = 1$, a zero-duration *dummy operation* O_{j,n_j+1} at v_0 is appended to each job, representing the return to the load/unload point after completion; in this way return moves and intermediate moves share the same task notation, and the makespan is always the completion time of each job's last operation.

Unlike in a vehicle routing problem, transport tasks are not part of the input; they are *induced* by the assignment:

$$\mathcal{T}(x) = \Big\{ (j,i) \;:\; d_{j,i-1} \neq d_{ji} \Big\}, \qquad d_{j0} \equiv v_0, \tag{1}$$

where task (j,i) moves job j from $d_{j,i-1}$ to d_{ji} and becomes ready at time $r_{ji} = C_{j,i-1}$ (with the convention $C_{j0} = 0$). If two successive operations are placed on the same arm, no transport task is created. Thus both the *number* and the *endpoints* of transport tasks are consequences of decision group (i): an assignment that keeps a job on the same arm throughout needs only two moves, whereas an assignment that changes arm at every operation needs $n_j + 1$ moves. Assignment of tasks to vehicles is indicated by $y_{tk} \in \{0,1\}$, with $\sum_{k\in K} y_{tk} = 1$ for each $t \in \mathcal{T}(x)$; tasks given to the same vehicle are ordered into a sequence.

In a constant travel-time matrix model, the assignment likewise determines the number of moves and their endpoints, but the duration of each move is a constant in the matrix, and corridor occupancy does not enter the model. In this paper, the same assignment also determines on which corridors those moves contend with one another.

3.4 Timing relations and feasibility

Write $p_{ji} = \sum_{m\in\Omega_{ji}} x_{jim} t^{P}_{jim}$ for the processing time on the chosen arm. Assignment, precedence, and the unit capacity of each arm give

$$\sum_{m\in\Omega_{ji}} x_{jim} = 1, \qquad \forall j,i, \tag{2}$$

$$C_{ji} = S_{ji} + p_{ji}, \qquad \forall j,i, \tag{3}$$

$$S_{ji} \;\geq\; A_{ji}, \qquad \forall j,i, \tag{4}$$

$$S_{j'i'} \;\geq\; C_{ji} \;\vee\; S_{ji} \;\geq\; C_{j'i'} \qquad \forall (j,i) \neq (j',i') \text{ on the same arm}, \tag{5}$$

where A_{ji} is the time at which the job actually appears at d_{ji}. If $(j,i) \notin \mathcal{T}(x)$, the job is already there and $A_{ji} = C_{j,i-1}$. Otherwise A_{ji} is produced by the vehicle that carries task (j,i): write a^{E}_{ji} for the time at which that vehicle reaches the pickup node after its empty leg; then

$$\ell_{ji} = \max\big(a^{\mathrm{E}}_{ji},\; r_{ji}\big), \qquad A_{ji} = \text{arrival time of the loaded leg departing at } \ell_{ji}, \tag{6}$$

i.e. the vehicle may wait for the job, and the job may wait for the vehicle. Whether empty or loaded, each leg is a walk $(e_1,\dots,e_q)$ in G whose corridor entry times $\eta_1,\dots,\eta_q$ satisfy

$$\eta_1 \;\geq\; \sigma, \qquad \eta_{l+1} \;\geq\; \eta_l + \tau_{e_l}, \quad l = 1,\dots,q-1, \tag{7}$$

where σ is the time at which the leg may start. If $\eta_{l+1} = \eta_l + \tau_{e_l}$, the vehicle enters the next corridor without stopping; if $\eta_{l+1} > \eta_l + \tau_{e_l}$, the difference is the vehicle's waiting time at a berth. The leg ends at $\eta_q + \tau_{e_q}$. Finally, the corridor-contention constraint is

$$\big[\eta,\; \eta+\tau_e\big) \;\cap\; \big[\eta',\; \eta'+\tau_e\big) \;=\; \emptyset \tag{8}$$

This constraint holds for any two distinct occupancies of the same corridor e, regardless of direction and of which vehicles are involved; it is also the only constraint in the model that couples different jobs through *space* rather than through a shared arm. The objective is

$$\min\ C_{\max}, \qquad C_{\max} = \max_{j \in J}\ C_{j,\,n_j+\delta_{\mathrm{ret}}}. \tag{9}$$

In a semi-active schedule (every operation starts at the earliest time the constraints allow), Eqs. (4) and (5) tighten to

$$S_{ji} = \max\big(A_{ji},\ F_{\mathrm{m}(j,i)}\big), \tag{10}$$

where F_m is the completion time of the operation sequenced immediately before O_{ji} on arm m; that is, the start time is the later of the job's arrival and the completion of the preceding operation on that arm. Section 4 uses this to attribute a postponed start to waiting for the workpiece or waiting for the arm.

3.5 Relation to FJSP_PT

Once Eq. (8) is removed, legs no longer occupy corridors against one another, so every leg can take a shortest path without waiting along the way. Transport durations collapse to the constant matrix $t^*(a, b)$, and the model becomes flexible job shop scheduling with transportation (FJSP_PT). The two problems therefore differ only in this set of corridor-exclusivity constraints: with them, the duration of a move is jointly determined by the vehicles' paths; without them, the duration is an input parameter. The resulting t^* is used later as a lower bound, and in the open-loop baseline to estimate arrival times by unimpeded travel time.

Relative to work that likewise supplies an explicit road network, the difference lies in what is reserved. van Os and Basten [21] reserve *nodes*: two vehicles may not occupy the same node at the same time, nor swap nodes. This paper reserves *edges*: corridors are exclusive in both directions, and at most one vehicle occupies a corridor at any time. Lyu et al. [13] likewise make corridors exclusive in both directions, but additionally allow each node to hold only one vehicle at a time; this paper instead provides berths of ample capacity at nodes for vehicles to wait in, consistent with how Assumption A4 treats arm buffers and the storage at v_0 and corresponding to the vehicle standby areas beside workstations in real shops, so congestion occurs only on corridors. The reasons why the instances cannot be compared directly are given under limitation (1) in Section 6.2.

3.6 Computational complexity

Proposition 1. *The problem defined by Eqs. (2)–(9) is strongly NP-hard, and it remains so when every operation satisfies* $\Omega_{ji} = M$.

Proof. Restrict to instances in which $\tau_e = 0$ for every $e \in E$, each robotic arm is connected to v_0 by a dedicated corridor, and $N_A = n$. Every leg then takes zero time and occupies the empty half-open interval $[t, t) = \emptyset$, so Eq. (8) holds trivially and no vehicle can delay another; moreover, because each job has a vehicle of its own, Eq. (6) never binds and $A_{ji} = C_{j,i-1}$. What remains is exactly Eqs. (2)–(5) with $A_{ji} = C_{j,i-1}$, i.e. the flexible job shop scheduling problem with unrelated machines. Two of its special cases are already strongly NP-hard: taking $|\Omega_{ji}| = 1$ everywhere reduces it to the classical job shop problem, which is strongly NP-hard already with three machines; taking $\Omega_{ji} = M$ and $n_j = 1$ reduces it to makespan minimization on unrelated parallel machines, which contains identical-parallel-machine scheduling and is strongly NP-hard when the number of machines is part of the input. Flexible job shop scheduling in general is itself strongly NP-hard [1]. Each of the restrictions above is a polynomial reduction, so the

general problem is strongly NP-hard; the second special case also shows that full flexibility does not make the problem easier. □

The reduction in Proposition 1 sets corridor traversal times to zero and thereby excludes vehicle contention. This shows that the problem is already strongly NP-hard even without corridor contention. If decisions (i)–(iii) are then fixed, the remaining path planning is still a conflict-free multi-agent routing problem [4], in which the release times and endpoints of the tasks are determined by the operation chains and the assignment and constrain one another. Conflict-free routing therefore adds constraints on top of the scheduling difficulty rather than replacing it. Exact methods already exist at small scale: van Os and Basten [21] use logic-based Benders decomposition to solve most benchmark instances under the same problem setting to proven optimality. This paper targets sizes beyond the practical reach of that exact decomposition.

3.7 Instance features

This paper holds instance size fixed and varies only structure; the effect of size itself is discussed under limitation (1) in Section 6.2. Each instance records the following features. The first four are quantities common in the scheduling literature; Xin et al. [22] note that $\bar{T}_t/\bar{T}_p$ among them is likely to affect algorithm performance.

- $\bar{T}_t/\bar{T}_p$: the mean shortest-path travel time between arm nodes divided by the mean processing time, i.e. the ratio of transport time to processing time.
- Heterogeneity H: the coefficient of variation of each operation's processing times over its optional arms, averaged over all operations (operations with $|\Omega_{ji}| = 1$ contribute 0). $H = 0$ means that the optional arms have identical processing times: reassignment does not affect processing time but can still be used to avoid congestion. As H grows, processing-time differences increasingly dominate the assignment and the room for reassignment to avoid congestion shrinks accordingly, so we do not expect the gain to rise with H; the measurements grouped by H in Section 5.4 agree with this.
- Flexibility $F = \mathrm{mean}_{ji}\, |\Omega_{ji}|/N_M$: the mean number of optional arms per operation as a fraction of all arms, i.e. the scope of reassignment.
- N_A/N_M: the vehicle-to-arm ratio. A smaller ratio makes the bottleneck more likely to lie in transport; a larger ratio, in processing.

We also record two quantities that are meaningful only on an explicit road network, describing the severity of congestion and the part of it that reassignment can avoid. Let $\mathrm{cut}(v_0)$ be a smallest set of corridors whose removal disconnects the load/unload point v_0 from every robotic arm.

- Contention intensity: on the initial random population, the share of total travel time that is yielding wait. It is determined by the instance structure rather than supplied as an input parameter; we use it to order the instances, and the per-instance values are given in Table 4.
- $|\mathrm{cut}(v_0)|$ (funnel width): an upper bound on the number of vehicles that can leave the load/unload point at the same time. The value 1 means that every vehicle must pass through the same corridor to leave. The layout family (Family A) varies contention by adding or removing such parallel exits; relative to `high`, `funnel` halves this value.

Each instance carries a composite lower bound $\mathrm{LB} = \max(\mathrm{LB}_{\mathrm{chain}}, \mathrm{LB}_{\mathrm{load}}, \mathrm{LB}_{\mathrm{cut}})$, computed directly from the instance data. The three components each relax a different part of the model, and none is always the largest on every instance. $\mathrm{LB}_{\mathrm{chain}}$ takes, for each job, "the shortest inbound trip + $\sum_i \min_{m\in\Omega_{ji}} t^P_{jim}$ + the shortest return trip", i.e. it

ignores inter-arm moves and all queueing. $\mathrm{LB}_{\mathrm{load}}$ takes $\sum_{j,i} \min_m t^P_{jim}/N_M$, i.e. it ignores operation precedence and transport. $\mathrm{LB}_{\mathrm{cut}}$ exploits the funnel: when $\delta_{\mathrm{ret}} = 1$ each job crosses $\mathrm{cut}(v_0)$ twice; spreading the $X = 2n$ crossings over the corridors of that cut so as to minimize the load on the busiest one yields

$$C_{\max} \;\geq\; X \Big/ \sum_{e \in \mathrm{cut}(v_0)} \tau_e^{-1}. \tag{11}$$

On the instances of this paper, the largest of the three is $\mathrm{LB}_{\mathrm{cut}}$ (on eight of the ten instances) or $\mathrm{LB}_{\mathrm{chain}}$ (on the other two), and $\mathrm{LB}_{\mathrm{load}}$ is never the largest—which agrees with an instance design that concentrates contention at the load/unload exit. The bound is fairly loose (its gap to the best known solution is 27% to 58%) and serves only as a correctness check: any solution below it indicates an implementation error (Section 5.1.2); it says nothing about how far a solution is from the optimum, which is discussed in Section 5.8 and under limitation (3) in Section 6.2.

4 Closed-loop bilevel framework

4.1 Overview: routing-in-the-loop evaluation and the decision path

The framework assigns the four groups of decisions of Section 3 to two levels. The input is an instance: the operation chains of the jobs, the option sets and processing times of the arms, the fleet size, and the corridor network. The upper level searches over assignment and sequencing with a memetic algorithm [23]; in the middle, an event-driven decoder translates one upper-level candidate into a sequence of precedence-constrained transport tasks; the lower level turns this sequence into conflict-free paths by time-window reservation. The output is an executable conflict-free timetable and its makespan. Relative to two-stage schemes or schemes that correct only once, the difference lies not in having two levels but in how the routing result enters the schedule: the content and direction of the feedback, and whether every candidate receives it.

The evaluation path, i.e. *routing-in-the-loop evaluation.* In every generation, the fitness of each candidate is the makespan after the lower level has completed conflict-free routing of all transport tasks, yielding wait included. The upper level takes this value as its objective, rather than the makespan computed on the assumption that vehicles never interfere and always travel unimpeded.

The decision path. Vehicle selection reads the current reservation table, instead of learning the outcome only after the paths have been generated. The mechanism is *reservation-table-aware dispatching*: for each candidate vehicle, the earliest feasible times of the empty leg and then the loaded leg are obtained by a trial query on the reservation table (the trial query does not write the final timetable), and the vehicle with the earliest feasible delivery is selected, rather than the vehicle whose arrival is earliest by the unimpeded-shortest-path estimate.

The contributions of the two paths to makespan are measured separately in Section 5. The two further inter-layer interfaces implemented are an extra cost on occupied corridor–time slots (hereafter, surcharge) and reassignment or resequencing from the yield record; their constructions are given in Sections 4.6 and 4.7, and the control results in Section 5.

The structure of the framework is shown in Figure 2. The two paths enter the upper level at different places, and the computational cost of a single evaluation therefore differs as well.

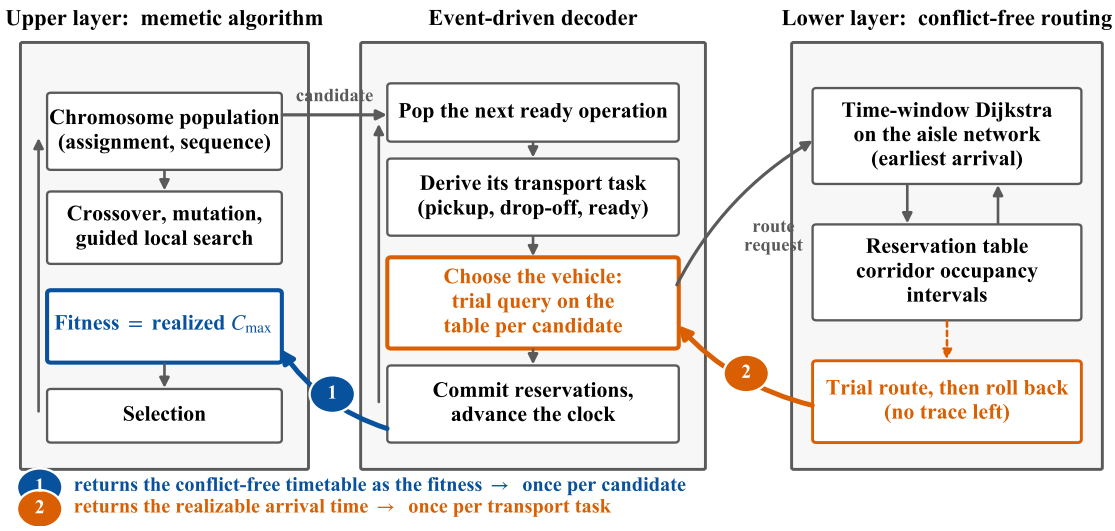


Fig. 2. The closed-loop bilevel framework. The upper level searches over assignment and sequencing with a memetic algorithm, the lower level performs conflict-free routing by time-window reservation, and an event-driven decoder in the middle translates one upper-level candidate into a sequence of transport tasks. The two feedback paths are distinguished by colour and enter the upper level at different places: path 1 (blue, marked 1) returns the conflict-free timetable completed by the lower level as fitness, once per candidate; path 2 (orange, marked 2) runs a trial query on the reservation table for the earliest feasible delivery time of the two legs before a vehicle is chosen, once per transport task; the trial query does not write the final timetable.

4.2 Lower level: time-window reservation and conflict-free routing

4.2.1 Reservation table. The only contended resource is the corridor (assumption A6), so the reservation table maps each corridor to a time-ordered list of half-open occupancy intervals $[t,\ t+\tau_e)$, each labelled with the vehicle and task that occupy it. The table supports four operations: query the earliest feasible entry time no earlier than a given instant; write an occupancy; revoke every occupancy of a given task; and save the current table state and restore it after a trial query (checkpoint and rollback). Nodes are not reserved: traversing a node takes zero time, and waits occur at berths (assumption A6 already separates berths from the traversal resource). Only one resource, the corridor, therefore needs to be reserved, and both the lower-level conflict-free path planner (hereafter, the router) and a checker independent of the decoder inspect only corridor occupancy.

4.2.2 Time-window Dijkstra. A leg is planned by a label-setting search. A label (u, t) means "standing at the berth of node u at time t", and its cost is t (Algorithm 1). When a corridor leaving the current node is examined, the reservation table is queried for the earliest feasible entry time $t' \geq t$; the gap $t' - t$ is the time the vehicle waits at the berth of u. The wait in Eq. (7) is therefore produced directly by the search and need not be modelled separately. Within one decoding, paths already written to the reservation table are not revoked, so vehicles are planned one after another in the fixed order in which the decoder generates tasks, and a later-planned vehicle yields to an earlier-planned one.

Lemma 1 (One earliest-arrival label per node suffices). *Keeping only the earliest-arrival label at each node is sufficient to obtain a path that reaches the destination earliest under the current reservation table.*

Algorithm 1 ROUTE: conflict-free planning of a single leg

Require: origin s, destination g, earliest departure time t_0, reservation table R, write-to-table flag *commit*
Ensure: a sequence of arrival time and corridor occupancy intervals; if *commit* is true, those intervals are also written to R

place the label (s, t_0) in a priority queue keyed by (time, node id) ▷ ties in time broken by node id, so the order is deterministic
while the queue is nonempty **do**
  extract the label (u, t) of least key
  if $u = g$ **then**
    backtrack to obtain the occupancy intervals; if *commit* is true, write them to R one by one; **return** arrival time t
  end if
  for all corridors $e = (u, v)$ leaving u **do**
    $t_{\text{in}} \leftarrow R.\text{EARLIESTENTRY}(e, t, \tau_e)$ ▷ wait at the berth of u until t_{in}
    if $t_{\text{in}} + \tau_e$ is strictly earlier than the incumbent arrival at v, update the label $(v,\ t_{\text{in}} + \tau_e)$
  end for
end while
return failure ▷ should not occur when G is strongly connected; see the text

PROOF. Berths have ample capacity and do not occupy the traversal resource (A6), so a vehicle that has stood at u since time t need only wait further until s in order to take any continuation that a vehicle arriving at u only at time $s > t$ could take. An earlier-arrival label is therefore never worse. Moreover, let $R.\text{EARLIESTENTRY}(e, t, \tau_e)$ denote the earliest entry time into corridor e that is no earlier than t and whose occupancy interval does not overlap any existing occupancy of e in the reservation table R. The later the query time, the fewer (never more) the still-feasible entry slots, so the time to finish traversing corridor e, $t \mapsto R.\text{EARLIESTENTRY}(e, t, \tau_e) + \tau_e$, is non-decreasing in t. The search extracts labels in increasing order of arrival time, and the first time the destination is extracted is the earliest arrival. □

Ordinary Dijkstra therefore suffices, and there is no need for the per-interval labelling of Safe Interval Path Planning (SIPP) [8]; if a second criterion is introduced (for example, a surcharge on corridor–time slots), earlier arrival is no longer always better, and a multi-label search is required (Section 4.6).

The number of reservations is finite, time is bounded, and G is strongly connected, so a path always exists: in the worst case the vehicle waits until every existing occupancy has ended and then travels along an unimpeded shortest path. This guarantees that Proposition 2 below holds, so the search need not repair infeasible solutions. A single call costs $O\big(|E|\,(W^2 + \log|V|)\big)$: at most $|E|$ corridors are examined; for each, $O(W)$ candidate entry times are enumerated, each candidate is tested for overlap in $O(W)$ time, and one heap operation of $O(\log|V|)$ is performed, where W is the mean number of reserved intervals per corridor. If the reservation table were replaced by an ordered structure supporting a logarithmic-time earliest-entry query, the complexity would drop to $O\big(|E|(\log W + \log|V|)\big)$; the implementation in this paper uses the linear scan above.

Running the same search on an empty reservation table over all pickup–delivery node pairs yields the unimpeded shortest-path matrix t^* of Section 3. It is used in three places: estimating arrival times by unimpeded time in open-loop configurations, the proximity term in the reassignment neighbourhood, and the transport times of the two-stage baseline; it is also the admissible lower bound used by the pruning of Section 4.5.

4.3 Upper level: encoding and event-driven decoding

4.3.1 Chromosome encoding. Let the total number of operations be N. An individual consists of two segments: a *machine-assignment vector* of length N, indexed by a fixed enumeration, whose k-th component is the robotic arm chosen for the k-th operation from Ω_{ji}; and an *operation sequence*, a permutation that allows a job to appear more than

Algorithm 2 DECODE: fitness from a complete conflict-free simulation

Require: instance, road network, assignment x, operation sequence
Ensure: makespan, timetable, per-corridor yield statistics
for every robotic arm set $F_m \leftarrow 0$; for every job set $\mathrm{pos}_j \leftarrow v_0$, $\mathrm{ready}_j \leftarrow 0$
for every vehicle set $\mathrm{loc}_k \leftarrow v_0$, $\mathrm{avail}_k \leftarrow 0$; clear the reservation table
for all operations O_{ji} taken in sequence order (dummy operations included) **do**
  $m \leftarrow$ the robotic arm of O_{ji} under x; $d \leftarrow \mathrm{loc}(m)$
  if $\mathrm{pos}_j = d$ **then**
    $A_{ji} \leftarrow \mathrm{ready}_j$ ▷ successive operations on the same arm; no transport task
  else
    choose vehicle k (Section 4.4)
    $a^{\mathrm{E}} \leftarrow$ ROUTE$(\mathrm{loc}_k, \mathrm{pos}_j, \mathrm{avail}_k, R, \mathrm{true})$; $\ell \leftarrow \max(a^{\mathrm{E}}, \mathrm{ready}_j)$ ▷ the empty leg occupies corridors as well
    $A_{ji} \leftarrow$ ROUTE$(\mathrm{pos}_j, d, \ell, R, \mathrm{true})$; $\mathrm{loc}_k \leftarrow d$; $\mathrm{avail}_k \leftarrow A_{ji}$
    accumulate yielding time by corridor
  end if
  if $F_m > A_{ji}$ then `bind` $\leftarrow$ `machine`, else `bind` $\leftarrow$ `arrive` ▷ record waiting for the arm or waiting for the workpiece
  $S_{ji} \leftarrow \max(A_{ji}, F_m)$; $C_{ji} \leftarrow S_{ji} + p_{ji}$; $F_m \leftarrow C_{ji}$; $\mathrm{pos}_j \leftarrow d$; $\mathrm{ready}_j \leftarrow C_{ji}$
end for
return $C_{\max} = \max_{j,i} C_{ji}$, the timetable, and the per-corridor yield statistics

once—job j appears n_j times, and its x-th appearance stands for O_{jx}. The sequence supplies a global order: the relative order of operations that share a robotic arm is read from it.

When $\delta_{\mathrm{ret}} = 1$, each job appears once more in the sequence, corresponding to the dummy operation of Section 3 (location fixed at v_0, processing time zero). Return transport therefore uses the same assignment and sequencing as intermediate transport, and the decoder needs no separate branch.

Which vehicle a task is given to is not written into the chromosome; it is decided at decode time (Section 4.4). A third segment for dispatching would enlarge the search space and would no longer guarantee that every chromosome is decodable, so a repair operator would be required.

4.3.2 Event-driven decoder. The decoder (Algorithm 2) implements the evaluation path. It sweeps the operation sequence from left to right, maintaining the idle time of each robotic arm, the current node and ready time of each job, and the current node and available time of each vehicle. When the arm of an operation differs from the job's current location, a transport task of Eq. (1) is generated, a vehicle is chosen, and two paths are planned against the *live* reservation table: an empty leg to the pickup point and a loaded leg to the destination, joined by Eq. (6). The operation then starts by Eq. (10); the decoder also records what determined the start—the arrival of the job (waiting for the workpiece) or the completion of the previous operation on that arm (waiting for the arm)—and later tracing of a postponed start takes this record as authoritative.

Decoding is deterministic: reservations are written in the order in which the decoder generates tasks, and ties in time are broken by a fixed rule at both sites—the routing label queue is keyed by (arrival time, node id) and a label is updated only when the new arrival is strictly earlier; dispatching scans vehicles in increasing id order and keeps the first strictly better one (Section 4.5). A chromosome therefore always corresponds to a unique timetable, and evaluation is deterministic. The seed-paired, equal-budget comparisons of Section 5 depend on this.

PROPOSITION 2 (EVERY CHROMOSOME IS DECODABLE). *For any machine assignment and any operation sequence, Algorithm 2 terminates and returns a conflict-free, deadlock-free timetable of finite makespan. The search therefore need not invoke a repair operator.*

Proof. By the encoding, the operation sequence is a topological order of each job's precedence constraints: the x-th appearance of a job stands for its x-th operation, so ready_j is already known when it is read. Ample buffers (A4) mean that an operation is never blocked for lack of storage, so once the job has arrived, Eq. (10) always supplies a finite start time. Each Route call returns in finite time by the argument following Algorithm 1, and occupies only slots that are still free on the reservation table, so Eq. (8) holds by construction. Deadlock-freedom likewise holds by construction: the corridor is the only contended resource (A6); waits occur only at berths of ample capacity and do not occupy corridors; every occupancy written to the reservation table ends at a finite time, so there is no circular wait among vehicles; in the worst case a vehicle waits until every existing occupancy has ended and then proceeds. The sweep executes $N + n\delta_{\text{ret}}$ times, and each time schedules a finite number of events at finite times. □

4.4 Reservation-table-aware dispatching

A vehicle may be chosen either by an unimpeded-shortest-path estimate or by a trial query on the reservation table. The open-loop rule scores each vehicle by

$$\text{score}(k) = \max\big(\text{avail}_k + t^*(\text{loc}_k, \text{pickup}),\ \text{ready}_j\big) + t^*(\text{pickup}, \text{destination}), \tag{12}$$

which does not consult the reservation table and may therefore select a vehicle that is geometrically near but whose corridors are already occupied. The formula is the same as the idle-first dispatching of Li et al. [10], and is also a typical single-attribute rule in surveys of multi-AGV dispatching [12].

The closed-loop default instead uses *reservation-table-aware dispatching*. For each candidate vehicle, the empty leg is planned and temporarily written to the reservation table, the loaded leg is then planned, the earliest feasible delivery time from this trial query is recorded, and the reservation table is rolled back to its state before the trial query. The empty leg must be written temporarily first; otherwise the loaded leg cannot see that occupancy, the two legs may be placed in the same time slot on the same corridor, and the trial result would be too early. The vehicle selected is the one with the earliest feasible delivery, not the one that is best by the unimpeded estimate of Eq. (12).

Vehicles have identical capability (assumption A5) and differ only in current location and corridor occupancy, so contention on the reservation table can directly determine which vehicle is selected.

The cost of the mechanism is that the trial query raises the routing calls per transport task from 2 to at most $2N_A$, so a single evaluation costs about 12.2 times as much as under rule-based dispatching. Unless this overhead is reduced, the makespan gain of the mechanism is offset by the extra computation under an equal wall-clock budget. The two speed-ups are given in Section 4.5; Section 5 therefore compares the two dispatching rules only under the same wall-clock limit.

4.5 Low overhead: admissible lower-bound pruning and winner-path reuse

Under an equal wall-clock budget, reservation-table-aware dispatching faces the following problem. At an equal number of *generations*, reservation-table-aware dispatching outperforms rule-based dispatching by 2.90% (Figure 7); but a single evaluation costs 12.2 times as much, so at equal *wall-clock time* this gain is offset by the extra computational overhead and the net effect is close to zero. Hence, as long as the cost multiple can be reduced without changing any selection, the gain of the mechanism becomes visible. This subsection gives two such speed-ups, both of which *provably do not change the selected vehicle or the timetable.*

Admissible lower-bound pruning. Conflict-free routing can only be *later* because of yielding; it can never be faster than the shortest path that ignores contention. The estimate of Eq. (12) used by the open-loop rule is therefore an *admissible lower bound* on the realized delivery time:

$$\mathrm{lb}(k) = \max\big(\mathrm{avail}_k + t^*(\mathrm{loc}_k, \mathrm{pickup}),\ \mathrm{ready}_j\big) + t^*(\mathrm{pickup}, \mathrm{destination}) \ \le\ \mathrm{est}(k), \tag{13}$$

where $\mathrm{est}(k)$ is the earliest feasible delivery time obtained by a complete two-leg trial query for vehicle k. Hence, if a vehicle's lower bound is already no earlier than the current best trial-query delivery time, routing need not be called for it.

Winner-path reuse. After the trial query the reservation table is rolled back as a whole, and the winner's two legs are discarded with it; the decoder then replans the winner on the same (restored) reservation table and writes the legs, causing two redundant routing calls. Since the search is deterministic, the replanned paths are exactly the same as those found in the trial query, so the paths computed for the winner during the trial query can be cached and written directly to the reservation table without recomputation.

Proposition 3 (The two speed-ups do not change the selected vehicle or the timetable). *In the implementation, vehicles are queried in increasing id order, and the selected vehicle is replaced only when some vehicle's feasible delivery is strictly earlier than the current best. Under this rule, after the pruning of Eq.* (13) *and winner-path reuse are added, the selected vehicle, the occupancies written to the reservation table, and the final timetable are the same as when a complete two-leg trial query is run for every vehicle.*

Proof. Consider pruning first. When vehicle k is reached, let the current best trial-query delivery be est^*. If $\mathrm{lb}(k) \ge \mathrm{est}^*$, then Eq. (13) gives $\mathrm{est}(k) \ge \mathrm{lb}(k) \ge \mathrm{est}^*$, so k's feasible delivery cannot be strictly earlier, and k would not be selected even if every vehicle received a complete trial query; not calling routing for it leaves the selected vehicle unchanged. The reservation table is saved before each trial query and restored afterwards; vehicles for which routing is not called do not rewrite the table and therefore do not affect later queries. Consider reuse next: the two legs of the selected vehicle were computed against one and the same reservation-table state, and the restored table is in the same state that the decoder faces when it subsequently writes. Since routing is deterministic, replanning in the same state yields the same paths as the cached ones, so writing the cached paths and replanning before writing produce identical occupancy intervals. □

Together the two speed-ups give a 2.33-fold speed-up, cut routing calls by 47.4%, and reduce the cost multiple of a single evaluation from 12.2 to 5.23. These three numbers are the *joint* effect of the two speed-ups: in the experiments they are measured as one on/off switch (Table 9), and are not attributed separately to pruning or reuse.

The effect of pruning varies with fleet size. The tighter the bound and the earlier a good trial-query delivery time is found, the more vehicles need no routing call; the larger the fleet, the more candidates there are, and the total trial-query cost grows approximately linearly with N_A. Measurements in Section 5 show that the pruning effect weakens as the fleet grows, but still cuts 42.2% of the routing calls on the largest fleet tested. Taken together, these effects mean that the gain of the mechanism may vanish beyond some fleet size; Section 5.4 explains that the present number of seeds is not sufficient to determine that size.

4.6 Two further inter-layer interfaces: surcharges and yield records

What the evaluation path and the dispatching trial query read from the lower level are both *realized durations*, i.e. traversal times after conflict-free routing. They rank every candidate by its makespan with its own yielding wait included; methods that schedule first and route once obtain a realized duration only once, after the assignment has been fixed. Realized durations, however, do not indicate which decision caused the wait. We therefore implement two further interfaces that carry richer information and compare them against it under an equal wall-clock budget; the results are in Section 5.5.

Return an extra cost on corridor–time slots. By analogy with the use of shadow prices to coordinate resources in Lagrangian relaxation and column generation, a surcharge can be attached to the contended resource: each corridor–time slot receives a surcharge $\pi(e, b)$, the amount that a vehicle occupying corridor e in time slot b must count in addition to its arrival time, so that path planning minimizes "arrival time $+\theta\cdot$ the sum of surcharges of the slots traversed" (the surcharge coefficient $\theta \geq 0$ is the only tunable parameter), and reassignment must likewise pay the surcharge for the traversal it requires. We implement two estimates: perturbing the capacity of a single slot and re-solving, and a low-cost approximation refreshed each generation.

Mechanistically, a surcharge provides no new information. The wait caused by contention already enters the vehicle's arrival time, then enters the makespan through Eq. (10), and is counted by the fitness; attaching a further cost to the same occupancy counts the same delay twice. Later tasks not yet planned may be blocked by reservations written earlier, but that effect is likewise already reflected in the makespan after a complete decode. Hence, as long as the fitness is taken from a complete conflict-free decode, attaching a congestion cost to the lower-level objective adds no information. We verify this inference only on this one interface.

The main overhead of a surcharge follows from Lemma 1: once the surcharge is a second criterion, earlier arrival is no longer always better, and single-label Dijkstra must be replaced by a multi-label Pareto search, so the cost of a single evaluation rises whether or not the surcharge is non-zero.

Return the yield record. The other choice is to return structure. When a yield occurs, path planning already knows the corridor, the time slot, and the transport that first occupied that corridor; the decoder keeps these yield events, not only the total wait. A record is a triple (corridor, time slot, preceding occupying transport), used to decide which operations local search should change. It does not enter any objective: the lower level still minimizes arrival time, the upper level still minimizes makespan; only the way the neighbourhood is generated changes. There is therefore no double-counting of delay. The operators are given in Section 4.7.

4.7 Yield-record-guided local search

This subsection writes the yield record of the previous subsection as local-search operators, applied to the top-ranked individuals in each generation; all four configurations of the main batch enable it with the same setting. Its critical-chain tracing is also used for the reading of the case study in Section 5.7.

Critical-chain tracing. Backtracking from the event that determines the makespan yields a chain. A chain that contains only operations cannot explain why a start was postponed: when the start is waiting for the workpiece, the postponement may come from an unfinished upstream operation, no vehicle being available, or corridor contention, and the three call for different changes. The backtrack therefore includes transport legs as well, and labels every link with a type (Table 3): a yield on a loaded leg is recorded as corridor occupancy; an empty leg is classified as "vehicle"

Table 3. Labels of the links on the critical chain, and the changes that can be taken. Only the corridor-occupancy class carries a concrete corridor and time slot, and only it can name the transport that first occupied that corridor.

Label	Criterion	Remedy
`operation`	a real operation is occupying its robotic arm	–
`machine`	waiting for the arm: blocked by the preceding operation on that arm	resequence
`upstream`	the vehicle arrived first, the material is not yet ready	recurse to $O_{j,i-1}$
`vehicle`	the material is ready, the vehicle is late, and yielding is a small share of the delay	change vehicle
`corridor`	a yield occurs on a specified corridor in a specified time slot	reassignment or staggering

or "corridor" by the share of yielding time in that leg's duration, with threshold 0.5. Only the corridor-occupancy class gives a concrete corridor, time slot, and preceding occupying transport, and this is exactly the information that local search uses.

Two families of operators and the acceptance rule. Reassignment: change the processing location. For at most $L = 5$ real operations O_{ji} on the critical chain (with current assignment on robotic arm m), each alternative robotic arm $m' \in \Omega_{ji} \setminus \{m\}$ is scored by

$$\text{score}(m') = t^*\big(d_{j,i-1}, \text{loc}(m')\big) + t^P_{jim'}, \tag{14}$$

i.e. the sum of the unimpeded travel time from the node $d_{j,i-1}$ of the previous operation to m' and the processing time on m'; the lowest-scoring arm produces a neighbour that changes only that one assignment. This allows a longer processing time to be traded for a shorter path with better traffic conditions, which is the reassignment trade-off of this paper. The score contains no cumulative-yield term, because that term is a total over the whole fleet and the whole planning horizon and is not commensurable with the scale of a single operation.

Staggering: change the order. Take the longest corridor-occupancy link on the chain and generate two directed neighbours: advance the blocked operation in the sequence, or postpone the preceding occupying operation, so that the two occupancies no longer overlap. A move only swaps adjacent appearances of different jobs, leaving the internal order of each job unchanged, so by Proposition 2 the result remains decodable.

The neighbourhood therefore has at most $L + 2 = 7$ candidates, each specified by the yield record. The first improvement is accepted, for at most three rounds; each round *recomputes* the critical chain from the current solution, because after a successful change the chain that determines the makespan often changes with it (Algorithm 3).

4.8 Outer genetic algorithm

The outer layer uses a conventional population search. The initial population is generated at random: a robotic arm is drawn uniformly from Ω_{ji}, and the sequence is shuffled; two further individuals are added, one assigning every operation to its fastest arm and one assigning greedily for load balance. Selection is by tournament with a small elite. For crossover, the sequence segment uses order crossover that preserves precedence, and the assignment segment uses uniform crossover; mutation either swaps two positions in the sequence or moves an operation to another optional robotic arm. Fitness is the decoded makespan. Hyperparameters are given in Section 5.1.3.

4.9 Complexity and cost

One fitness evaluation processes $N + n\delta_{\text{ret}}$ events. Under closed-loop dispatching each transport uses at most $2N_A$ routing calls, so each individual costs $O\big((N+n)\,N_A\,|E|\,(W^2 + \log|V|)\big)$; open-loop selection by unimpeded time no longer carries the factor N_A. The two speed-ups of Section 4.5 change neither this worst-case bound nor the selected

Algorithm 3 ContentionGuidedSearch: yield-record-guided local search

Require: chromosome c and its decode result r
Ensure: a chromosome whose makespan is no worse than $r.C_{\max}$, and its decode result

for *round* = 1 **to** 3 **do**
  $\mathcal{N} \leftarrow \textsc{Reassign}(c, r) \cup \textsc{Stagger}(c, r)$ ▷ at most $L + 2$ neighbours, each specified by the yield record
  improved ← **false**
  for all $c' \in \mathcal{N}$ **do**
    $r' \leftarrow \textsc{Decode}(c')$
    if $r'.C_{\max} < r.C_{\max}$ **then**
      $c \leftarrow c'$; $r \leftarrow r'$; *improved* ← **true**; **break**
    end if
  end for
  if not *improved* **then**
    break
  end if
end for
return c, r

vehicle or the timetable (Proposition 3); measured, they cut routing calls by 47.4% in total (Section 5; between 42.2% and 61.2% across instances), which amounts to replacing N_A by a smaller effective multiple. Local search runs at most three rounds with at most $L + 2$ neighbours per round, so it adds at most $3 \times (L + 2) = 21$ further decodes for each selected individual.

Within the framework, the cost of a single evaluation can differ by about two orders of magnitude across configurations. On the ladder batch the measurements are: open-loop 0.38 ms, closed-loop rule-based dispatching 2.70 ms, and full closed-loop 14.2 ms, the closed loop being about 37 times the open loop. The surcharge is not in that batch; it comes from an earlier surcharge scan, within which turning the surcharge on raises a single evaluation from 15.05 ms to 72.9 ms, a further factor of about 4.8. The absolute timings of the two batches are not directly comparable, but the product of the two multiples shows that the configurations with the highest and lowest single-evaluation cost differ by about two orders of magnitude. Equal generation counts are therefore not an appropriate comparison, and Section 5 aligns wall-clock time instead.

Under a fixed budget, a more expensive single evaluation means fewer evaluations can be completed. Whether an interface converts into a makespan gain cannot be deduced from its design in advance; it must be measured under an equal wall-clock budget (Section 5).

The results above guarantee correctness, not solution quality: Proposition 2, Lemma 1, and Proposition 3 guarantee that every candidate is feasible, that its evaluation is consistent with the conflict model of Section 3, and that the two speed-ups do not change any selection; together with the deterministic decoding of Section 4.3, the solution procedure is reproducible. As with most metaheuristics, solution quality has no theoretical bound and is evaluated experimentally in the next section.

5 Experiments

This section answers four questions. How large is the gain from including yielding wait in the fitness (the closed loop)? How much further does reservation-table-aware dispatching reduce makespan, can it replace the closed loop, and do the two amplify each other when used together? Over what range of instances do these gains hold, and *can the present number of seeds determine* that range? Why do the other interfaces that we implemented fail to yield a net gain?

5.1 Experimental setup

5.1.1 Controlled instance families. FJSP_PT benchmarks usually write travel times as a matrix indexed by location pairs; it records only the total duration between endpoints and supplies no corridor network. Of the 7 layouts used by Homayouni and Fontes [6], 7 matrices are entirely asymmetric (consistent with one-way tracks), and 2 of them also violate the triangle inequality. Asymmetry rules out undirected corridors of the kind used here (one τ_e per corridor, so the resulting t^* must be symmetric); violating the triangle inequality is stronger still: the all-pairs shortest paths of any graph satisfy the triangle inequality, so these 2 matrices cannot be recovered even if directed graphs are allowed. A network reverse-engineered from such a matrix would be a different instance, and the published reference values would no longer be comparable. We therefore generate a set of controlled instances: the road network is given explicitly, and each feature of Section 3.7 can be varied on its own. A separate comparison on public instances—with the contention constraint removed, so that the problem reverts to the same mathematical problem as in the literature—is given in Section 5.8; it calibrates the strength of the upper-level search, not our framework.

All instances share the same core: 12 jobs, 3 operations per job for 36 real operations, 8 robotic arms, heterogeneity $H = 0.3$, and a transport-to-processing ratio $\bar{T}_t/\bar{T}_p$ scaled to 4.0, so that vehicle contention is the main constraint. H is 0.3 on all ten instances and does not vary with flexibility. For each operation, the generator first draws independent perturbations of its $|\Omega_{ji}|$ processing times, standardizes them to zero mean and unit variance, and then multiplies them by H; the within-row coefficient of variation is therefore independent of $|\Omega_{ji}|$, and changing F in Family C does not change H. Small deviations caused by rounding are measured after generation and recorded with each instance. On this core, the instances vary along three factors (Table 4). Every instance is generated with a fixed random seed and is fully reproducible.

Family A: layout (contention structure). The first three instances use a spine layout: a main corridor (the spine) runs through the arm area, and each arm hangs off it on its own branch corridor at one of two hubs on the spine, half near the load/unload point v_0 and half at the far end of the spine; v_0 sits at one end of the spine and connects to it through one or two parallel exits. This matches the spine layout common in high-volume plants—processing areas line both sides of a central corridor, in-plant moves travel along it, and congestion on that corridor is a known shop-floor problem [18]. Real shops usually place the store or load/unload point where the trips to all stations are roughly balanced, to shorten transport and spread corridor occupancy; we place v_0 at one end of the spine, so every transport must pass the same exit and the same stretch of the main corridor. This is a deliberately high-contention case, and it is exactly the case the closed-loop mechanism is meant to handle. We also choose the spine layout because it concentrates contention on parallel lanes that can be added or removed: adding a lane changes capacity only, not distance. A tandem single-vehicle loop keeps vehicles from meeting each other [17]; it has almost no contention and does not suit the purpose of this study.

The spine layout covers only the high-contention range. Covering the low-contention range needs another topology, the *grid*: with more intersections and more detours, an occupied corridor can be bypassed and contention falls, but the trip from the load/unload point to the farthest arm becomes longer. The five instances therefore split into two topologies: `funnel`, `high`, and `mid` use the spine layout; `low` and `scatter` are grids. Contention intensity broadly decreases in this order (31.9% to 6.6%) but not strictly: the grid layout `low` has slightly higher contention (11.8%) than the spine layout `mid` (10.8%), because the names denote the levels intended at design time; measured values are in Table 4. The paper-wide maximum contention, 33.5%, is not in this family. It belongs to the instance that raises flexibility to $F = 1.0$ (every operation may use every arm) on the same `funnel` layout, which is part of Family C below.

The three spine instances differ only in their parallel lanes ($1 \to 2$ exits at the load/unload point, $1 \to 2$ lanes in the middle segment). Each lane is a two-hop path of equal total duration, so adding or removing one changes only how many vehicles can pass in parallel, not the distance; the node/corridor counts rise from 13/12 to 15/16. `funnel` and `high` differ only by one parallel exit at the load/unload point, which halves the minimum cut; they form a single-factor contrast. The two grid instances switch to a 4×4 grid (16/24), and to a scattered layout on the same grid that places the arms at the farthest points. The two have the same size and edge weights and differ only in arm placement; v_0 again sits at a corner.

The unimpeded trip from the load/unload point to the farthest arm is 10.0 on all three spine instances and 18.0 on both grid instances, nearly twice as long. The two grid instances also have the highest makespans in the family, even though their contention intensity is the lowest. This family is therefore a splice of "capacity + topology", not an end-to-end single-factor scan: differences among the three spine instances can be attributed to capacity, while crossing over to the grid changes capacity and distance together. When Section 5.4 orders the instances by contention intensity, two inversions appear: one stems from this topological splice, and the other lies inside the spine layout and needs a separate explanation. Contention intensity is the quantity with which we directly describe congestion, and this family spans the widest range of it, so that section's judgement on where the closed loop applies rests mainly on this family.

Family B: vehicle-to-arm ratio N_A/N_M. On `funnel`, take $N_A \in \{4, 8, 12, 16\}$, i.e. ratios of 0.5, 1.0, 1.5, and 2.0; 1.5 is the base point shared by the three families (listed as `A funnel` under Family A in the table). This family tests whether dispatching has an upper limit in fleet size (Sections 5.4 and 5.6).

Family C: flexibility F. On the same layout, take $F \in \{0.3, 0.6, 1.0\}$, i.e. the fraction of arms each operation may use; 0.6 is again the base point. $|\Omega_{ji}|$ is randomized and rounded so that its expectation equals $F \cdot N_M$; the measured base point is 0.61 (Table 4 lists measured values; the other two levels match their targets). Flexibility sets the scope of reassignment, and also how far corridor contention can be turned into an adjustment of the assignment.

The three families comprise 10 instances. The base point `A funnel` is at once the 1.5 of Family B and the 0.6 of Family C; it is generated and run only once, and occupies a single row in Tables 4 and 8. Hence $5 + 3 + 2 = 10$, while the Family B and Family C scans have four and three levels, respectively. Section 5.4 must count the base point when it reads these two families; leaving it out would make them look more monotone than they are.

Contention intensity is determined by the instance structure. It is measured on the initial random population (20 individuals) as the mean share of yielding wait in travel time; per-instance values are in Table 4. We do not use optimized solutions, because they reflect the solution rather than the instance: the search routes around congested corridors, contention tends to zero after convergence, and the differences among instances disappear. Section 5.4 uses this quantity to order the instances.

Comparison with public instances. As an external reference for the H, F, and $\bar{T}_t/\bar{T}_p$ of our generated instances, we use the 20 public instances compiled by Homayouni and Fontes [6] (also used for the degenerate comparison in Section 5.8); both sets of features are computed with the same feature function and convention, so they can be compared directly. Median flexibility is 0.38; our base point of 0.6 is about one and a half times that, and the $F = 0.3$ of Family C falls inside the public range; median heterogeneity is only 0.135, the largest of the 20 instances is 0.258, and none reaches our $H = 0.3$; the public maximum of $\bar{T}_t/\bar{T}_p$ is only 0.12, while we scale it to 4.0.

The last two are both extrapolations, but with different meanings. $\bar{T}_t/\bar{T}_p = 4.0$ is deliberate: if transport is only a few percent of processing time, vehicle contention need not be modelled separately, and the common FJSP_PT instances are of this kind. $H = 0.3$ lies above the literature range; if H raised the closed-loop gain, part of the differences among

Table 4. Controlled instances. All instances share 12 jobs, 36 real operations, 8 robotic arms, $H = 0.3$, and $\bar{T}_t/\bar{T}_p \approx 4.0$; the three families vary only the layout, the vehicle-to-arm ratio, and flexibility, respectively. Contention intensity is the share of yielding wait in total travel time on the initial random population; it is determined by the instance structure and is not an input parameter.

Instance	Varied factor	Nodes	Corridors	N_A	F	Contention	Composite LB (time units)
A funnel	layout	13	12	12	0.61	31.9%	24.0
A high	layout	14	14	12	0.61	30.9%	17.0
A low	layout	16	24	12	0.61	11.8%	36.0
A mid	layout	15	16	12	0.61	10.8%	17.0
A scatter	layout	16	24	12	0.61	6.6%	36.0
B NA/NM 0.5	N_A/N_M	13	12	4	0.61	15.8%	24.0
B NA/NM 1.0	N_A/N_M	13	12	8	0.61	30.2%	24.0
B NA/NM 2.0	N_A/N_M	13	12	16	0.61	32.4%	24.0
C F=0.3	flexibility	13	12	12	0.30	27.7%	24.0
C F=1.0	flexibility	13	12	12	1.00	33.5%	24.0

configurations would come from this value. Section 5.4 tests this with independent data spanning four levels of H, and the gain does not rise with H; the instances and baselines of that batch differ from the main batch, so it supports only this directional conclusion.

5.1.2 Comparison protocol. The comparison rests on the following rules.

Common executor: report the realized makespan. The makespan that the open loop schedules from unimpeded travel times is often infeasible on the real corridors. Every configuration's schedule is therefore executed by the same conflict-free router and then checked by a validator independent of the decoder; what is compared is always the realized makespan after execution. The gap between the open loop's self-reported and realized values (hereafter, the optimism margin) is especially large on the funnel layout. If each configuration were compared on its own self-reported value, the open loop would gain an advantage from a makespan that cannot be realized.

Equal wall-clock budget, not equal generation count. A single evaluation does different work in each configuration: the open loop looks up the constant matrix t^*; the closed loop runs conflict-free routing for every candidate; the surcharge interface additionally needs a multi-label search. The cost of a single evaluation can differ by about two orders of magnitude. Comparing at an equal generation count would give the more expensive configuration more compute, and an apparent mechanism gain might be only a larger budget. We therefore align wall-clock time: all configurations on the same instance share the same budget, 90 seconds in the main batch, with a cap of 2000 generations and early stopping after 400 consecutive generations without improvement. The generation cap never fires. Every open-loop run is ended by early stopping after a little over four hundred generations, using on average about one fifth of the budget; both closed-loop configurations use the full 90 seconds every time. This setting favours the baseline: the open loop is not cut off by the budget but searches until it stops improving, and in its shorter wall-clock time it completes as many evaluations as closed-loop rule-based dispatching, or more (Section 5.6). Each remaining batch answers one specific question; their instances, seeds, and budgets are summarized in Table 5.

Aligning wall-clock time lets the configuration with the cheaper evaluation complete more evaluations. Every configuration therefore reports both its cost per evaluation and the number of evaluations actually completed, and Section 5.6 analyses the trade-off between the two.

Table 5. Experiment batches. The main batch supports every statistical conclusion in Tables 7 and 8; each other batch answers one specific question and is not set side by side with the main batch instance by instance. "Speed-ups" states whether the pruning and winner-path reuse of Section 4.5 had been implemented. The surcharge mechanism and the yield-record-guided local search did not change before and after the speed-ups, and their cost is not affected by the pruning.

Batch	Purpose	Instances	Seeds	Wall-clock budget	Speed-ups
Main	Four-level ladder (Tables 7, 8)	Ten controlled instances	10	90 s	yes
Cost diagnosis	Cost per evaluation (Figure 6)	Three controlled instances	3	90 s	yes
Convergence diagnosis	Convergence curves (Figure 3)	`funnel`	3	46.3 s	yes
Speed-up switch	Table 9	Six controlled instances	10	90 s	off/on
Budget convention	Figure 7	Four instances	10	Equal wall-clock and equal generations	no
Surcharge scan	Figure 5	Two $8 \times 4 \times 4$ instances	10/5	48.9/60.7 s	no
Independent instances	Cross-check, H, ablations (Sections 5.4, 5.5)	16 instances	10	30.2–72.9 s	no
Public benchmarks	Table 10	20 public instances	10	Early stopping	—

Seeds and paired tests. A single seed cannot support a conclusion: on the same instance and configuration, three seeds produced a range of 11 makespan units, larger than any mechanism effect being measured. Each instance is therefore run with 10 seeds, giving $10 \times 10 = 100$ pairs in total, and a two-sided Wilcoxon signed-rank test is performed with pairing by seed. 10 seeds are enough to resolve an effect of a few percentage points when the 100 pairs are pooled, but not inside a single instance. Section 5.4 shows that this boundary falls exactly between the closed loop and dispatching; per-instance conclusions are confined to this power.

Multiple comparisons and effect sizes. The paper reports a little over twenty Wilcoxon tests. Without correction, the chance of "at least one significant instance out of ten" is about 40% under the global null, not the nominal 5%. Every family of tests is corrected within that family, and the family is stated where the result is reported. There are two kinds of family: the five pooled contrasts of Table 7 form one family; in Table 8, the 10 instances of each effect form one family. We use the Holm correction to control the chance of at least one Type I error within a family; Benjamini–Hochberg is computed as well, and any case where the two disagree is marked where it occurs.

Except in two places, all p-values in this paper are corrected; both places are marked as uncorrected where they appear. The first is the four p-values for the vehicle-to-arm ratio in Section 5.4. The claim there is that the effects are *not* significant, and correction can only increase a p-value: a value not significant before correction remains not significant after it. Judging non-significance on the uncorrected values is therefore the stricter standard, and it also shows how far each value lies from the threshold. The second is the count of significant instances among the 16 independent instances cited in Sections 5.4 and 5.5. Their instances and baselines differ from those of the main batch, and the counts are taken from that batch's own report; they serve only to corroborate direction and magnitude and are not the test basis for any conclusion, so they are not re-corrected by this paper's families.

Correction only answers whether an effect is significant, not how large it is. Each pooled contrast also reports a Hodges–Lehmann pseudomedian with a 95% distribution-free confidence interval. These come from the same source as the Wilcoxon signed-rank test: the point estimate, the interval, and the p-value are all read from the same null

distribution. The mean pairwise relative gain, the main figure in our tables, is convenient for adding across tables, but it does not come from the rank test and carries no precision. We give both. For example, the mean for dispatching inside the closed loop is 2.45% and the pseudomedian is 3.03%; the gap comes from pairs on the two vehicle-to-arm-ratio instances where makespan worsens (positive gain), and the pseudomedian is insensitive to such values. Error bars in the figures are the standard error of the mean pairwise relative gain, shaded bands are the seed-to-seed range; tables give no ±, since precision is conveyed by the HL confidence intervals and the significance marks.

Correctness. For each solution, a validator independent of the decoder reconstructs the corridor occupancies from the timetable and compares the result with the composite lower bound. Runs that fail the check or fall below the bound are reported separately and are not included in any instance mean.

5.1.3 Implementation and environment. The experiments run single-threaded on an Intel Xeon W-1270 @ 3.40 GHz. The implementation is pure Python ($\geq$ 3.9.6) using only the standard library, with no third-party dependencies and no solver. The hyperparameters of the outer genetic algorithm and the local search take their defaults, are not tuned, and are identical across configurations: population 60, crossover probability 0.8, mutation probability 0.2, elite 5, binary tournament; the local search acts each generation on the top 10% of individuals, examines at most $L = 5$ critical operations per round, and runs at most three rounds. Because the hyperparameters are conventional and identical across configurations, the measured gains come from the mechanisms rather than from operator tuning. Absolute runtimes cannot be compared with those of a compiled implementation. A faster implementation would let every configuration complete more evaluations within the same budget, but would not change the ratio of cost per evaluation between configurations; this ratio is measured between configurations that share the same implementation.

5.2 Four-level baseline ladder

The comparison must answer two questions separately: how much the makespan differs when the structure is aligned with the pipeline structures common in the literature, and how much it differs when only one component is changed. The former requires structural correspondence; the latter requires changing one thing at a time. Ablations alone cannot be mapped onto the common pipeline structures, while a literature comparison alone would credit the gain of conflict-free routing itself to our mechanisms. We therefore use a four-level ladder in which each level corresponds to a clearly identifiable structure.

B0: open-loop planning + realized execution. The upper level searches by unimpeded shortest-path times (the search ignores conflicts); after the search, the plan is executed by the same conflict-free router. This represents the open-loop structure: the scheduling layer produces a plan that ignores transport contention, and a downstream conflict-free router executes it. FJSP_PT methods that schedule on a constant matrix [7, 14, 16] fall into this class when executed on a real road network, as does the framework of Li et al. [10]. Dispatching is idle-first, the same as Eq. (12) used by rule-based dispatching in our framework.

B0+: open-loop planning + reservation-table-aware dispatching. The schedule is likewise fixed by unimpeded travel times; only vehicle selection is changed to a trial query on the reservation table. This represents the two-stage structure: the first stage schedules on a *constant travel-time matrix*, and only the second stage optimizes vehicle assignment and handles conflicts on the fixed schedule, with no feedback (e.g. Sun et al. [19]). Here, reservation-table-aware dispatching serves as a greedy approximation of that second stage.

Table 6. The 2 × 2 structure of the four-level ladder. Rows: whether the search includes conflict-induced delay; columns: whether a vehicle is selected by unimpeded time or by a trial query on the reservation table. Comparisons within a row or a column differ in one factor; a diagonal comparison changes both and cannot be decomposed.

	Rule-based dispatching	Reservation-table-aware dispatching
Open loop (schedule first, then route; search ignores conflicts)	B0	B0+
Closed-loop planning (fitness from conflict-free routing)	B1	**B2** (ours)

B1: closed-loop planning + rule-based dispatching. Fitness during the search is taken from conflict-free routing, while dispatching still estimates times from unimpeded shortest paths via Eq. (12). It differs from B2 only in the vehicle-selection rule.

B2: closed-loop planning + reservation-table-aware dispatching. Our method.

The four levels are in fact a 2×2 design along two factors, not a linear sequence. The first factor is whether the search includes conflict-induced delay (open-loop / closed-loop); the second is whether a vehicle is selected by unimpeded time or by a trial query on the reservation table (rule / trial query). If two configurations differ in both factors at once, their difference cannot be decomposed. For example, B0+ → B1 crosses both factors and **cannot** be used to measure the closed loop. The interpretable contrasts fix one factor and change only the other, i.e. the four pairs that share a row or a column of Table 6.

How B0 is executed decides how strong the baseline is. The plan found by the open-loop search already contains a set of vehicle assignments, and it can be reproduced at execution time in two ways: *replay the plan* executes with the vehicles the original plan specifies; *on-site reassignment* re-selects vehicles by the rule at execution time. The former is measured to be stronger (63.00 versus 78.00 on the funnel instance) and matches the practice of Li et al. [10], whose task-allocation layer fixes the robots on the Gantt chart first and, at execution, only shifts the time axis without reassignment. The main report uses the stronger replay-the-plan convention; on-site reassignment is listed alongside.

5.3 Aggregate comparison on the ladder

Table 7 gives the makespans of the four-level ladder over 10 instances × 10 seeds, i.e. 100 pairs in total, with the same wall-clock budget on each instance (Section 5.1.2); every run passed the independent check. The next three paragraphs read the effect along each of the two factors, and then the pooled B0 → B2. The p-values in the table are Holm-corrected over these five contrasts, and the correction changes no decision. The text reports pseudomedians and confidence intervals as *reductions*; Table 7 shows the same intervals as negative values.

Closed-loop evaluation. Fixing the vehicle-selection rule and changing only whether the search includes conflict-induced delay, the closed loop is significantly better under both dispatching rules: under rule-based dispatching, B0 → B1 is -18.86% ($p < 10^{-4}$); under reservation-table-aware dispatching, B0+ → B2 is -17.75% ($p < 10^{-4}$). For the former, the Hodges–Lehmann pseudomedian is 18.28% with 95% confidence interval $[14.68\%,\ 21.95\%]$. The whole interval lies far from zero and even its lower end is a two-digit reduction, so this level does not depend on whether the point estimate is a mean or a pseudomedian. The gap cannot be explained by the number of evaluations, because that difference runs the other way: an open-loop evaluation only looks up a constant matrix, so within the same budget the open loop completes several times as many evaluations as the closed loop (the cost per evaluation of each configuration is shown in Figure 6). The open loop evaluates more and is still worse, because it optimizes a surrogate objective that excludes yielding; the true transport cost appears only after the plan is fixed. Figure 3 shows this gap on a single instance.

Table 7. Makespans of the four-level ladder under an equal wall-clock budget, and the four pairs that change only one thing along the two factors. A negative value means a lower makespan, i.e. better. $n = 100$ comparisons paired by (instance, seed), two-sided Wilcoxon signed-rank tests; p-values are Holm-corrected over the five contrasts of this table ($^{*}p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$). The HL column is the Hodges–Lehmann pseudomedian of pairwise relative gains with its 95% distribution-free confidence interval (see Section 5.1.2). A bold header marks our configuration; numbers are not bold because B2 is not best on every instance (per-instance results in Table 8).

		$C_{\max}$ (time units)			
Instance	Contention	B0	B0^{+}	B1	**B2**
A funnel	31.9%	58.10	53.50	44.70	42.40
A high	30.9%	56.00	51.40	39.90	37.70
A low	11.8%	64.50	64.60	57.80	57.10
A mid	10.8%	46.40	42.70	39.40	37.40
A scatter	6.6%	62.30	62.30	57.20	57.10
B NA/NM 0.5	15.8%	71.00	70.50	69.00	65.10
B NA/NM 1.0	30.2%	56.70	56.50	47.50	49.80
B NA/NM 2.0	32.4%	70.60	64.20	42.20	44.20
C F=0.3	27.7%	74.30	69.50	51.60	47.30
C F=1.0	33.5%	52.10	47.50	39.30	36.30
Pooled mean	—	61.20	58.27	48.86	47.44

Single-factor paired contrast	$\Delta C_{\max}$	HL pseudomedian and 95% CI (%)	W/L/T	p_{Holm}
Closed-loop value (rule-based)	$-18.86\%^{***}$	$-18.28\ [-21.95, -14.68]$	96/2/2	$< 10^{-4}$
Closed-loop value (reservation-table-aware)	$-17.75\%^{***}$	$-18.00\ [-20.55, -15.22]$	93/5/2	$< 10^{-4}$
Dispatching value (open-loop)	$-4.59\%^{***}$	$-4.45\ [-6.13, -2.88]$	68/19/13	$< 10^{-4}$
Dispatching value (closed-loop)	$-2.45\%^{**}$	$-3.03\ [-5.19, -0.94]$	60/34/6	0.0054
End-to-end: literature → this paper	$-21.35\%^{***}$	$-21.20\ [-24.46, -17.99]$	94/5/1	$< 10^{-4}$

The closed loop is significant under both dispatching rules, so this conclusion does not depend on our vehicle-selection rule.

Reservation-table-aware dispatching. Fix the open or closed loop and change only the vehicle-selection rule. *Inside the closed loop*, B1 → B2 is -2.45% ($p = 0.0054$), with pseudomedian 3.03% and 95% confidence interval $[0.94\%,\ 5.19\%]$. The two levels share the same scheduling search and the same budget; they differ only in whether vehicle selection uses a trial query on the reservation table. This is a pooled result. The interval does not cross zero, but its lower end is less than one percentage point from zero; after family-wise correction no instance is significant (2 before correction), and two of the ten instances have a positive sign, i.e. B2 is slower than B1 at vehicle-to-arm ratios 1.0 and 2.0 (Table 8). Thus -2.45% is an average effect over this instance family and cannot be applied instance by instance; Section 5.4 discusses this resolution limit.

Under the open loop, the same rule gives -4.59% ($p < 10^{-4}$), likewise significant. It measures the step from the open-loop structure to the two-stage structure; its magnitude (4.59%) is far smaller than that of the closed loop (18.86%). Reservation-table-aware dispatching is significant under both the open and the closed loop, but its effect is much smaller and cannot by itself explain the B0 → B2 reduction.

End to end, and how the two effects combine. B0 → B2 is -21.35% when pooled ($p < 10^{-4}$; of the 100 pairs, 94 wins, 5 losses, and 1 tie). That is, moving from open-loop planning with rule-based dispatching to closed-loop planning with

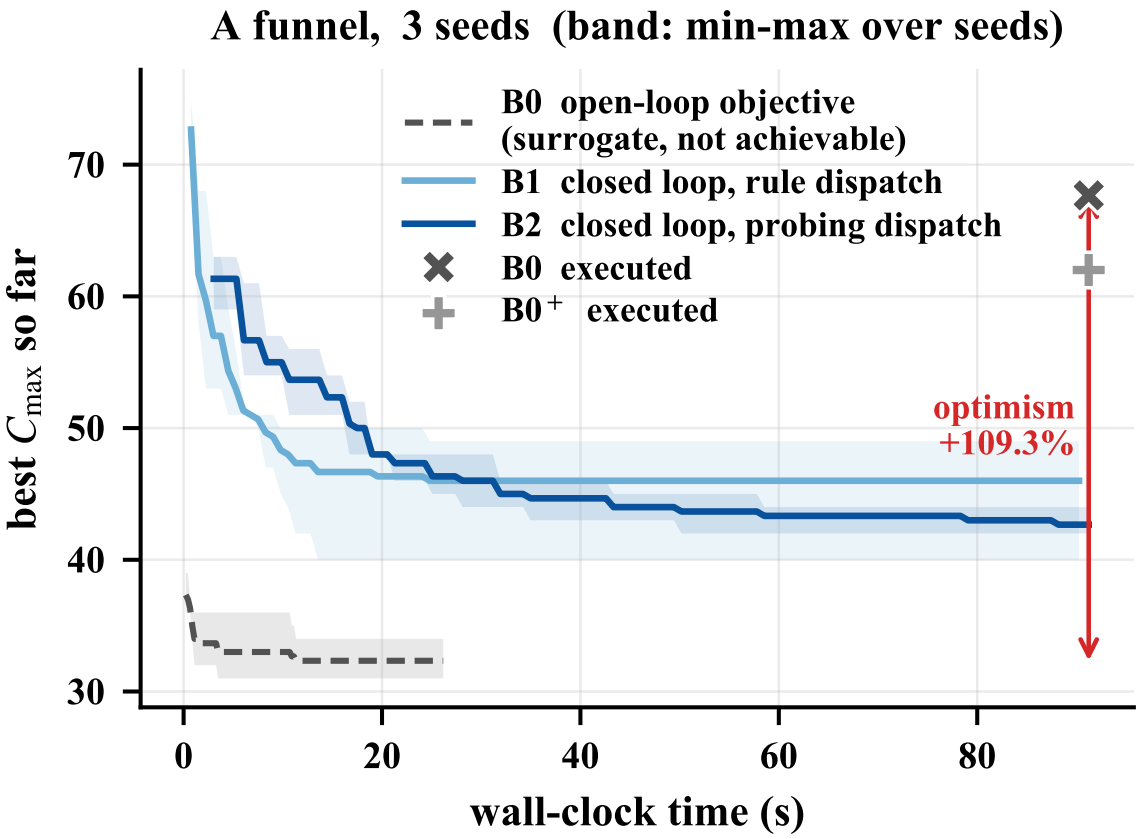


Fig. 3. Convergence curves on a single `funnel` instance, for illustration only (taken from a diagnostic batch of three instances; see Table 5). The horizontal axis is wall-clock time; each curve is the time-wise mean over three seeds, and the shaded band is the seed-to-seed *range* (the sample is too small to estimate quartiles). The solid lines (B1, B2) are always realized makespans; the dashed line is the surrogate objective of the open-loop search, i.e. the quantity it actually optimizes; crosses and pluses mark the makespan of the same plan after execution by conflict-free routing. The gap between the end of the dashed line and the cross is the optimism margin on this instance: the open loop has not failed to converge; it has converged to a surrogate objective that cannot be realized.

reservation-table-aware dispatching lowers makespan by about one fifth. The three effects have different reference points, so they cannot simply be added. If the closed loop and dispatching were independent, the end-to-end gain would equal the compound of the closed loop under rule-based dispatching (18.86%) and dispatching under the open loop (4.59%), i.e. 22.6%. The measured 21.35% is slightly below this, which shows that each mechanism contributes on its own and *the two do not amplify each other*: B2 is best end to end because the two gains add up.

Relation to "schedule first, then resolve conflicts". Dispatching still yields a significant 4.59% under the open loop, so the shortfall of this approach lies not in second-stage dispatching but in the first stage ignoring yielding. Placed in a two-stage structure, the same dispatching rule gains only as much as open-loop dispatching, far less than the closed loop's 18.86%; to obtain the closed-loop reduction, yielding wait must enter the fitness during the search. These gaps were measured on our instance families.

5.4 Three-factor scans and per-instance resolution

The aggregate comparison of Section 5.3 rests on 10 instances. This section takes them apart by layout, vehicle-to-arm ratio, and flexibility. The trend of the closed loop can be read by family; dispatching is significant on no single instance. This section reports that difference. Each of the three families corresponds to an expectation that can be stated in advance.

Layout (Family A) tests whether the gain grows with contention intensity: both paths target the delay caused by vehicles yielding to one another, and when contention is weak this delay is small, so their effect is limited.

Vehicle-to-arm ratio (Family B) tests whether there is an upper bound on fleet size beyond which overhead offsets the gain. Trial-query cost grows approximately linearly with the fleet, and the pruning of Section 4.5 has its lowest cut rate on the instance with the largest fleet.

Flexibility (Family C) tests whether the gain depends on room for reassignment.

Per-instance resolution. The significance marks of Table 8 decide which instances may be read by sign. If each of the 10 instances is tested once, the probability of "at least one significant" under the global null is far above 5%. Per-instance tests therefore receive a Holm correction by family, the family being the 10 instances of one effect (the closed loop and dispatching each form a family; they answer different questions and are not merged into one family of twenty tests). The marks in Table 8 and Figure 4 are all based on the corrected p.

After correction the closed loop remains significant on 9 instances (9 before correction; the largest corrected value is $p = 0.0401$, so the verdict does not depend on the correction). The only exception is the instance with the fewest vehicles (B0 → B1 is -2.0%, not significant). The trend of the closed loop can therefore be read by family. Dispatching inside the closed loop is significant on 2 instances before correction (the two ends of the flexibility family, $p = 0.0105$ and $p = 0.0321$), and *on 0 instances after correction*: Holm gives $p = 0.1052$ and $p = 0.2885$, and Benjamini–Hochberg gives $p = 0.1052$ and $p = 0.1603$, likewise not significant. No instance reaches significance on its own. The effect size (a few percentage points) is of the same order as the smallest effect that 10 seeds on one instance can resolve. Only the 100 pooled pairs reach significance, and only just: the pseudomedian is 3.03%, and the 95% interval $[0.94\%,\ 5.19\%]$ does not cross zero, with its lower end less than one percentage point from zero. Below, per-instance results for dispatching are reported as directions only; the sign is judged only on the pooled data.

Table 8 and Figure 4 give the per-instance results. The three families are read differently.

Layout family. The closed loop rises roughly with contention intensity and is significant on all five instances: on the least congested `scatter` (contention 6.6%), B0 → B1 is -8.1%; on `mid`/`low` it is -14.4%/-10.3%; on `funnel`/`high` it is -21.3%/-26.6%. We say "roughly" because there are two inversions of different kinds. `low` has higher contention than `mid` but a smaller gain. The two lie on opposite sides of the topological boundary of Section 5.1 (grid versus spine, farthest trip 18.0 versus 10.0), so this is not a pure contention inversion. `funnel` has higher contention than `high` but a smaller gain. Both are spine instances that differ only by one parallel exit at the load/unload point, the planned single-factor contrast, so *this inversion is a counter-example.* What can be established is the magnitude (-21.3% on the most congested instance versus -8.1% on the least congested, about two and a half times), not a monotone relation between contention and gain. Contention intensity summarizes congestion as a single number and does not separate the part that reassignment can bypass; see Limitation (1) in Section 6.2. Dispatching is not significant on any instance of this family, so only its direction is reported: four of the five instances are negative, the same direction as the closed loop.

Vehicle-to-arm-ratio family. This family does not establish an upper bound of applicability for dispatching. Dispatching inside the closed loop at the four levels is, in order, −4.7% ($N_A/N_M = 0.5$), +5.0% (1.0), -4.6% (1.5, the base point

Table 8. Per-instance results of the four-level ladder and the two effects, closed loop and dispatching, blocked by the three families (each instance has $n = 10$ comparisons paired by seed). Significance marks follow the **Holm-corrected** p (two-sided Wilcoxon signed-rank test), the family being the 10 instances of one effect: $^{*}p < 0.05$, $^{**}p < 0.01$; † means significant before correction but not after, and the sign of that instance is not taken as a conclusion. A negative value means a lower makespan. The two gain columns use the same estimator as Table 7 (the mean of pairwise relative gains over seeds) and cannot be obtained by dividing the two means in the same row. `A funnel` is at once $N_A/N_M = 1.5$ of Family B and $F = 0.6$ of Family C; it is run only once and occupies only one row.

		$C_{\max}$ (time units)				Pairwise relative gain	
Instance	Contention	B0	B0^{+}	B1	**B2**	B0→B1	B1→B2
A funnel	31.9%	58.10	53.50	44.70	42.40	−21.3%*	−4.6%
A high	30.9%	56.00	51.40	39.90	37.70	−26.6%*	−4.8%
A low	11.8%	64.50	64.60	57.80	57.10	−10.3%*	−1.1%
A mid	10.8%	46.40	42.70	39.40	37.40	−14.4%*	−4.8%
A scatter	6.6%	62.30	62.30	57.20	57.10	−8.1%*	+0.1%
B NA/NM 0.5	15.8%	71.00	70.50	69.00	65.10	−2.0%	−4.7%
B NA/NM 1.0	30.2%	56.70	56.50	47.50	49.80	−15.6%*	+5.0%
B NA/NM 2.0	32.4%	70.60	64.20	42.20	44.20	−37.7%*	+5.7%
C F=0.3	27.7%	74.30	69.50	51.60	47.30	−29.7%*	−8.2%†
C F=1.0	33.5%	52.10	47.50	39.30	36.30	−22.9%*	−7.2%†

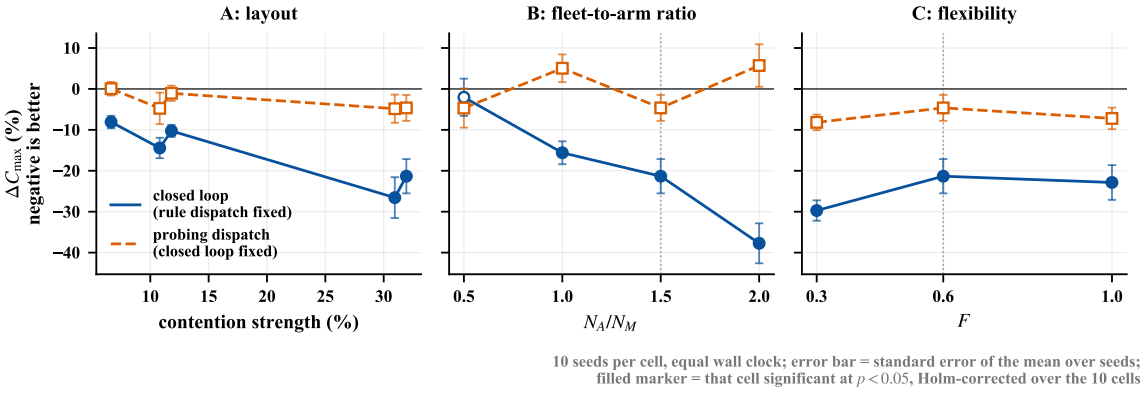


Fig. 4. The closed loop and dispatching across the three families. Left: layout family, horizontal axis contention intensity; middle: vehicle-to-arm ratio, N_A/N_M; right: flexibility. Families B and C include the shared base point (1.5 and 0.6, at the hollow vertical lines), which is also listed as `A funnel` in Family A. Filled markers remain significant after Holm correction ($p < 0.05$, the family being the 10 instances of one effect); hollow markers are not significant. Error bars are the standard error of the mean of pairwise relative gains over the 10 seeds of each instance. The closed loop is significant on 9 instances and rises roughly with contention; after correction dispatching is significant on *no instance*, its error bars cross zero on every instance, and its sign alternates across the four levels of the vehicle-to-arm ratio. The point of the figure is whether the two curves deviate significantly from zero, not the shape of the dispatching curve.

`A funnel`), and +5.7% (2.0). The sign alternates, and none of the four instances is significant ($p = 0.4121$, $p = 0.2011$, $p = 0.1386$, $p = 0.3313$; these are values *before* correction, already non-significant). Taking only the three levels 0.5, 1.0, and 2.0, the curve shows the monotone rise predicted by the cost law; once the base point in the middle is included,

this trend no longer holds. The upper bound predicted in Section 5.6 therefore remains untested: testing it would require enough seeds to resolve effects of a few percentage points, which exceeds the budget of this paper. The closed loop at 1.0 and 2.0 is -15.6% and -37.7%, both significant: fleet size mainly affects the closed-loop gain, not dispatching.

Flexibility family. After Holm correction, the dispatching effect is not significant in this family either. At $F = 0.3$, 0.6, and 1.0, dispatching is -8.2%, -4.6%, and -7.2%. The three have the same sign, and the two ends are the two largest magnitudes among all 10 instances (the base point falls back to a level comparable with the rest). The raw values at the two ends, $p = 0.0105$ and $p = 0.0321$, rise after correction to $p = 0.1052$ and $p = 0.2885$, which are no longer significant. This family provides only directional evidence, of the same sign as the pooled result and of the largest magnitude. The same sign at all three levels, without the sign alternation of the vehicle-to-arm-ratio family, is compatible with "dispatching acts mainly through vehicle selection and timing, and does not depend strongly on the number of eligible arms": if dispatching relied on room for reassignment, $F = 0.3$ should fail first, yet it is the instance with the largest magnitude. The closed loop is -29.7% and -22.9%, still significant per instance after correction.

An independent batch of self-generated instances. The results above come from one instance family and one baseline. As a cross-check we use another batch of self-generated instances with contention (public benchmarks are treated in Section 5.8, where contention has been removed): 16 instances × 10 seeds, 1280 runs in all. Both the instance core and the baseline differ: the baseline is two-stage surrogate transport, and the batch crosses four contention levels with four heterogeneity levels. The *integration gain* (two-stage → closed-loop) is 5.52%, significant on 11 of the 16 instances and ranging from 0.28% to 9.28%, consistent in direction and magnitude with the closed loop in this section. Counts of significant instances in this batch are uncorrected (see Section 5.1.2).

Heterogeneity does not raise the closed-loop gain. The four H levels of that batch answer the question left open in Section 5.1. Grouping the 16 instances into four levels by H (four instances per level, one at each contention level), the integration gain is 6.12% at $H = 0$, 5.62% at $H = 0.15$ (measured 0.139), 5.95% at $H = 0.3$, and 4.38% at $H = 0.5$; the rank correlation between gain and H is a weakly negative -0.21.

The measured $H = 0.139$ lies near the public-instance median 0.135, and its gain 5.62% differs from the 5.95% at $H = 0.3$ by less than half a percentage point: setting H above the literature range did not raise the closed-loop gain. The expectation that the gain rises with H does not hold: the trend is not monotone, and the rank correlation is negative. Each level has only 4 instances, and the within-level spread (0.28% to 9.28%) is far larger than the gaps between levels. The data therefore support only "the gain does not rise with H" and are not sufficient to assert that $H = 0.5$ is lower. This is compatible with the analysis of Section 3.7: as H grows, processing-time differences among eligible arms gradually outweigh differences in corridor accessibility, so assignment is decided more and more by processing speed and less and less by traffic conditions. $H = 0$ removes the trade-off, not the opportunity: all arms in Ω_{ji} have the same processing time, so reassignment need not trade processing time for travel time and becomes a pure congestion decision at zero cost. This level has the highest gain of the four (6.12%), consistent with the definition. Thus H turns arm selection into a trade-off (without heterogeneity, the reversal of Section 1 would not occur), but it does not amplify the closed-loop gain. The closed-loop gain comes from whether the search accounts for yielding; contention strength is set by layout and fleet, not by H.

For reservation-table-aware dispatching, that batch gives -0.74% (0 instances significant), the opposite direction from the main batch. The batch ran before the two speed-ups; see Section 5.6 for the reason.

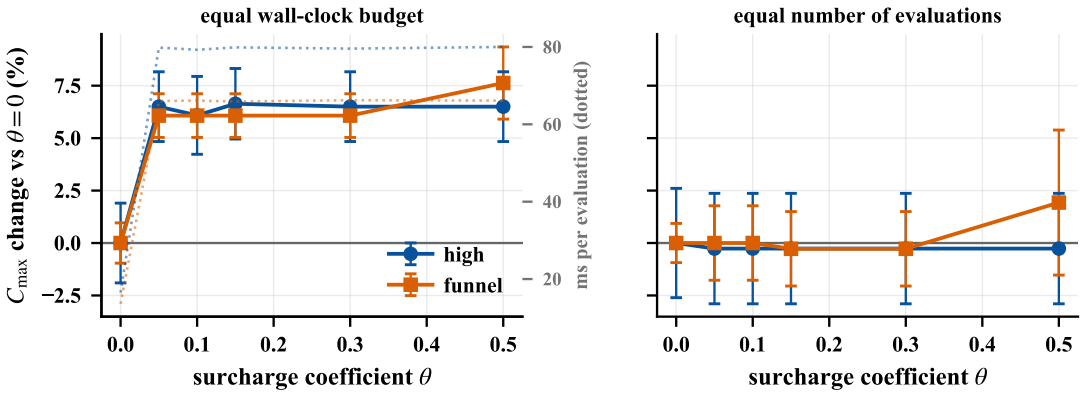


Fig. 5. Surcharge on corridor–time slots. A positive value means worse. Under equal wall-clock time (left), the makespan worsening appears as soon as the surcharge is turned on, then stays flat over a ten-fold range of θ, in step with the cost per evaluation on the right axis; under an equal number of evaluations (right), the worsening disappears. The damage comes from a higher cost per evaluation and hence fewer evaluations within the same budget, not from the surcharge choosing wrong paths. Markers are means and error bars are the standard error of the mean; the number of runs per point is given in the figure footer. "Significant" refers to a two-sided Wilcoxon signed-rank test of each θ against $\theta = 0$, paired by seed. The two instances do not belong to the three families of Table 4, and the legend labels `high`/`funnel` are layout tags only; see Table 5 for the batch.

5.5 Ablations and controlled negative controls

This section reports measurements of the two interfaces of Section 4.6. The surcharge is off by default ($\theta = 0$). Yield-record-guided local search is enabled with the same setting in all four configurations of the main batch. It makes no stable contribution to makespan (see the ablation below), but because it is identical across configurations, it does not affect the comparisons between them.

5.5.1 Yield-record-guided local search. On the 16-instance batch of Section 5.4, turning off yield-record feedback gives a relative gain of -1.72% (significant on only 1 of the 16 instances), and turning off the staggering operator gives -0.16% (significant on 2 instances). Neither is positive: under an equal wall-clock budget, decodes spent on this neighbourhood are better spent on population evolution. The mechanism is explained in Section 5.5.3.

These two ablations may cite that earlier batch, but the dispatching ablation may not. The yield record and staggering were not changed before or after the two speed-ups, and their cost does not vary with pruning; the cost of dispatching is exactly what the speed-ups target.

5.5.2 Surcharge on corridor–time slots. Section 4.6 pointed out that adding a surcharge to already occupied corridor–time slots counts again a delay that fitness already includes. That argument shows that the surcharge adds no information, but it cannot give the size of the makespan loss, which must therefore be measured separately. We scan θ under both budget conventions and fix the search width of path planning at every θ, so that the effect of the surcharge is not confounded with "how many entry times are examined".

The measurements are shown in Figure 5. Under equal wall-clock time, every θ tried makes the makespan worse, and significantly so; the worsening is flat over a ten-fold range of θ. Under an equal number of evaluations, the same

comparison is not significant. Routing with a surcharge completes about one fifth as many evaluations within the same budget as routing without it: the surcharge does not mis-rank candidate solutions, but it sharply reduces the number of evaluations within the same budget.

Two further observations rule out other explanations. First, θ is not an effective tuning parameter on this problem. Over two orders of magnitude, routing with a surcharge modifies the same few paths, and both the cumulative surcharge and the returned makespan stay the same. The cumulative surcharge is only a small fraction of total travel time, not enough to change the order of arrival times in arrival time + $\theta \cdot$ surcharge. Only when θ is so large that the surcharge outweighs the arrival time does the double counting take effect, and then makespan worsens sharply. Second, the overhead is structural and is not caused by a non-zero surcharge. The runtime ratio of the same decode with and without a surcharge is the same at every θ; the extra cost comes from "maintaining a Pareto front at every node and trying several entry times on every arc", regardless of whether a surcharge is actually applied. This agrees with Lemma 1: once a second criterion is introduced there is no longer a total-order dominance, and single-label search must be replaced by multi-label search.

The two budget conventions disagree on the size of the loss but agree on its sign. Whether the surcharge comes from a finite difference of corridor capacity or from a critical-chain proxy, the delay it targets is already counted in the arrival time of the yielding vehicle. As long as fitness is taken from a complete conflict-free decode, the surcharge carries no new information, and the comparison is decided by the overhead of the multi-label search it requires. Hence, before extending the inter-layer interface, one should first check whether the objective of the receiving layer already contains the quantity the interface would pass; if it does, such an extension is harmful under our protocol.

5.5.3 Dilution and overhead. Why the yield record and staggering fail to offset their overhead can be established by measurement; the same measurement also explains why the overhead of reservation-table-aware dispatching had to be reduced first.

Allocation of decodes. Yield-record-guided local search consumes a large share of all decodes, yet fewer than one in fifty of the neighbours it decodes improve makespan. These improvements are real, but the same decodes spent on population evolution yield more.

The neighbourhood is small and well targeted by construction (Section 4.7). A low improvement rate despite good targeting means that the operations named by the record do not shorten the critical chain after reassignment or staggering. We tried three refinements: scoring reassignment by the expected yielding wait along the path rather than by unimpeded shortest-path time; replacing the adjacent swap of staggering with a directed insertion; and accepting plateau moves that "leave makespan unchanged but lower total yielding wait". Each was compared independently with the original operator under an equal number of evaluations, and *none* was significant. The failure therefore lies in the signal itself, not in how the signal is used.

Dilution along the critical chain. A yield record names the corridor and the time slot of a yield and the transport that occupied that corridor first, that is, the information needed to remove *that* event. Makespan is determined by a chain. Once one corridor delay is removed, another constraint (an upstream operation, a robotic arm, or another corridor) usually becomes tight, so the realized improvement is only part of the delay removed. This is why the acceptance rule re-extracts the critical chain after every acceptance. When a bottleneck is only one of several of comparable magnitude, feedback that names it improves makespan by far less than the yielding time it removes.

Why dispatching gives different results before and after the speed-ups. The same dilution acts on reservation-table-aware dispatching. The trial query selects the vehicle that delivers the current task earliest, which is a local improvement; only part of it survives when it propagates through the whole critical chain to makespan. After dilution the reduction is still positive, but only a few percentage points. Before the speed-ups, a single evaluation cost 12.2 times as much as under rule-based dispatching, too few evaluations fit into the same wall-clock time, and the net effect was not significant (the independent batch of Section 5.4). Only after Section 4.5 lowered the cost to 5.23 times did this reduction become significant in the pooled data.

Under a fixed budget, two quantities that can be measured and improved separately must be considered together: the makespan reduction that remains after dilution, and the extra cost per evaluation. The two mechanisms fail for different reasons. Yield-record-guided local search is limited by the former: too little reduction survives dilution, and none of the three neighbourhood refinements reaches significance under an equal number of evaluations. Dispatching is limited by the latter: the post-dilution reduction is always present, and it was initially not significant because the cost per evaluation was too high, a cost that can be lowered without changing the selected vehicle or the timetable. This paper therefore lists dispatching as a mechanism contribution and reports yield-record-guided local search only as a controlled negative control.

5.6 The cost law and the effect of the two speed-ups

Under a fixed wall-clock budget, the cost per evaluation and the number of evaluations trade off against each other. The same dispatching rule has a net effect that is not significant before the speed-ups and significantly positive after them, so this section measures both quantities directly.

Two budget conventions. Figure 6 shows the cost per evaluation of each configuration and the number of evaluations actually completed within the same budget cap. The cost spans about two orders of magnitude: cheaper configurations complete more evaluations, and the open loop completes the most evaluations while using only about one fifth of the wall-clock time. Comparing at an equal number of generations lets the more expensive configuration use more compute; aligning wall-clock time lets the cheaper configuration complete more evaluations. We therefore report both conventions.

Figure 7 runs the same set of comparisons once under each convention. The equal-generation convention ignores the extra cost per evaluation and therefore favours the configuration with the higher cost per evaluation: reservation-table-aware dispatching relative to rule-based dispatching is 2.90% ($p = 0.0269$). Under equal wall-clock time, before the speed-ups, the same comparison gives only 0.03% ($p = 0.8799$), close to zero. The equal-generation convention asks whether the extra routing computation improves the ranking in each generation; the equal-wall-clock convention asks whether makespan falls within the same time. Both questions are meaningful, but neither can stand in for the other.

The cost law. Reservation-table-aware dispatching raises the number of routing calls per transport task from 2 to at most $2N_A$, so its cost grows linearly with fleet size. The lower bound used by the pruning is tighter when contention is weaker, so more candidates are skipped; the larger the fleet and the stronger the contention, the smaller the cut. The equivalence self-check of Section 4.5 agrees with this direction: among its six instances, the highest cut rate in routing calls is on the least congested one (61.2%) and the lowest on the one with the largest fleet (42.2%); the pruning weakens but does not fail. Together, the two trends *predict* that when the fleet is large enough and contention strong enough, the growing trial-query cost will offset the gain. In Section 5.4, the sign alternates across the four levels of the vehicle-to-arm ratio and none of the four instances is significant, so this upper bound is not confirmed. The cost law

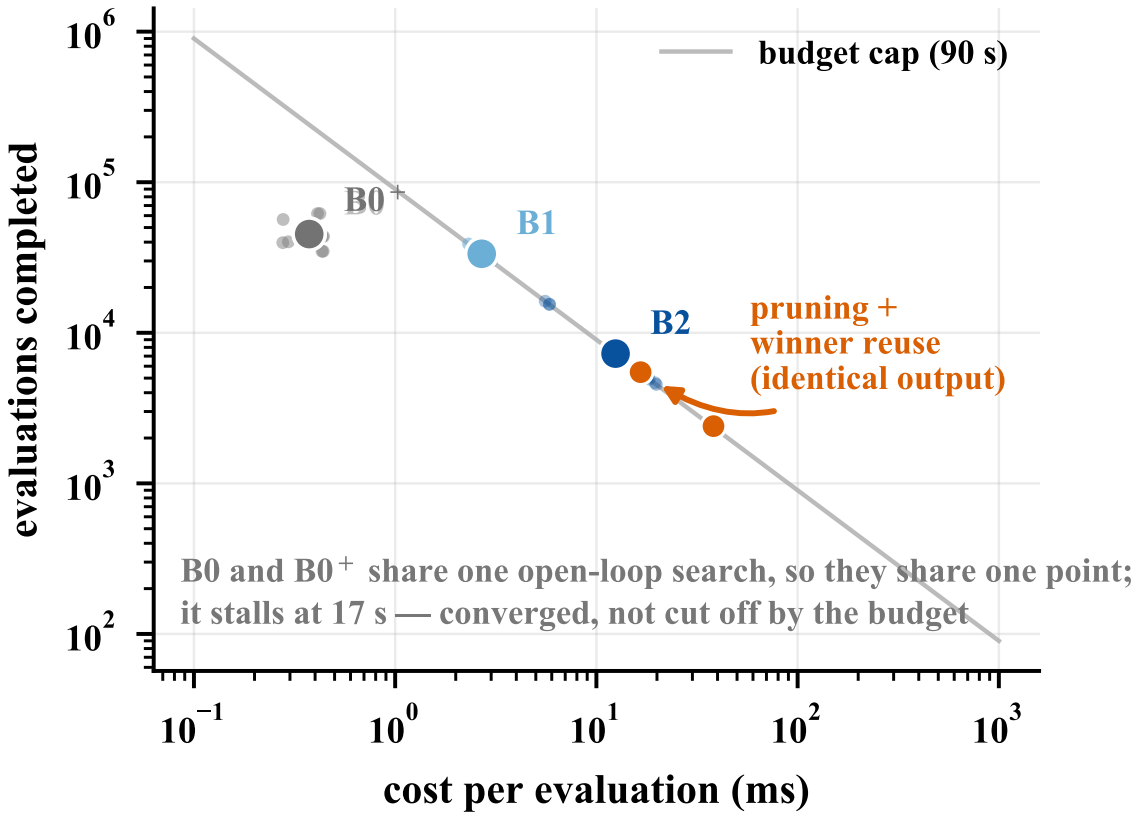


Fig. 6. A fixed wall-clock budget places the cost per evaluation and the number of evaluations completed on a hyperbola; the grey line is the 90-second cap shared by all configurations. A point on the line means that cost × count equals that configuration's wall-clock time. The two closed-loop configurations lie on the line, that is, every run uses the full budget. The open loop lies *below* it: it stops improving after a little over four hundred generations and is ended by early stopping, converging in about 17 seconds rather than being cut off by the budget (Section 5.1.2). B0 and B0$^+$ share the same open-loop search and therefore share one point; they differ only in how the schedule is executed. Pale small points are the individual measurements on three instances, and the large filled point is their geometric mean: on a log axis, only the geometric mean preserves "cost × count = that configuration's wall-clock time". Orange markers are the off and on settings of pruning and winner-path reuse (the callout marks the "on" setting). By Proposition 3 the two produce identical output, so the displacement along the line reflects only the compute saved.

itself (cost growing linearly with N_A, pruning effect falling with N_A) is measured directly; the upper bound derived from it has not yet been verified.

Switching the two speed-ups on and off. The speed-ups and the dispatching rule were introduced one after the other, so setting the two batches side by side cannot rule out the effect of differences in instances and baselines. We therefore run a separate control that changes only the speed-ups: same instances, same seeds, same budget, with pruning and winner-path reuse switched on or off.

Proposition 3 guarantees that the two speed-ups *do not change the selected vehicle or the timetable*. On each of six instances we take six random chromosomes; with the speed-ups on and off, the makespans and dispatch sequences are *identical bit for bit* in all 36 cases, without exception, and routing calls fall by 47.4% in total. Since the output

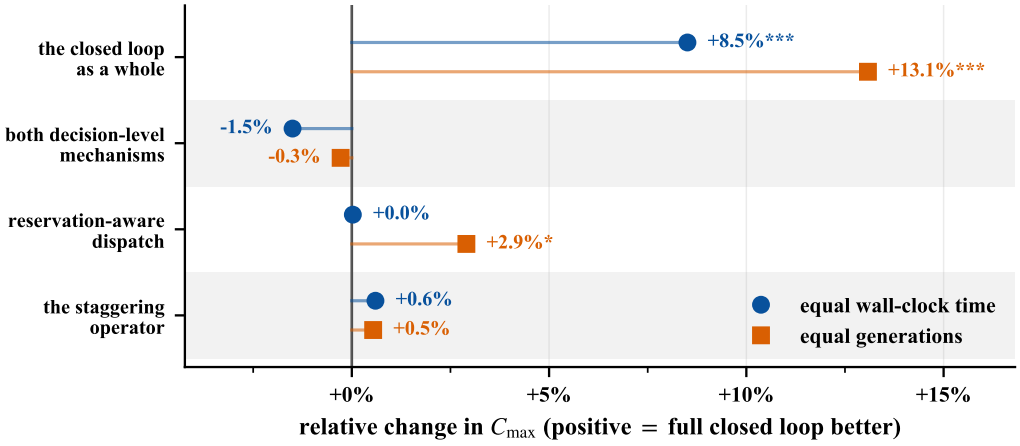


Fig. 7. The same set of comparisons under the two budget conventions. Each row is named by the mechanism it isolates. The difference between the two markers comes from the convention: reservation-table-aware dispatching is 2.90% ($p = 0.0269$) at an equal number of generations and 0.03% ($p = 0.8799$), close to zero, at equal wall-clock time. The batch ran before the two speed-ups (Table 5), which is also why the speed-ups are switched on and off in a separate control below.

Table 9. Changing only pruning and winner-path reuse. "Off"/"on" indicates whether the two are enabled; the bold header marks the setting adopted in this paper. The two settings produce bit-identical output (verified by the equivalence self-check), so the difference comes only from the compute saved. Same instances, same seeds, same wall-clock time; $n = 60$ pairs, two-sided Wilcoxon signed-rank test ($^{***}p < 0.001$); the aggregate row lists wins/losses/ties and p. Both ratio columns use the *off* value as denominator, in opposite directions: evaluations as on/off, time per evaluation as off/on.

		C_{max} (time units)			Eval. count ratio	Speedup per eval.
Instance	Contention	Off	**On**	ΔC_{max}	(on/off)	(off/on)
A funnel	31.9%	44.80	42.10	−6.03%	2.21×	2.23×
A high	30.9%	41.60	37.70	−9.37%	2.40×	2.41×
A low	11.8%	60.50	57.00	−5.79%	2.79×	2.81×
B NA/NM 0.5	15.8%	66.70	65.10	−2.40%	2.11×	2.11×
B NA/NM 1.0	30.2%	51.90	48.90	−5.78%	2.31×	2.31×
B NA/NM 2.0	32.4%	44.50	42.20	−5.17%	1.98×	2.01×
Pooled (60 pairs)	—	—	—	$-5.60\%^{***}$		50/0/10, $< 10^{-4}$
HL pseudomedian and 95% CI: -4.72 [$-6.34, -3.55$] %						

is identical, turning the speed-ups off only makes each evaluation slower, and any makespan difference under equal wall-clock time comes *only* from the compute saved.

Table 9: going from off→on lowers makespan by a further 5.60% (50 wins / 0 losses, $p < 10^{-4}$); the number of evaluations within the same budget is 2.23 times as large, and a single evaluation runs 2.33 times faster. This control links the two batches: the same rule gives -0.74% on the matrix batch before the speed-ups (0 of the 16 instances significant) and is significantly positive on the ladder batch after them, while the speed-ups themselves change no selection.

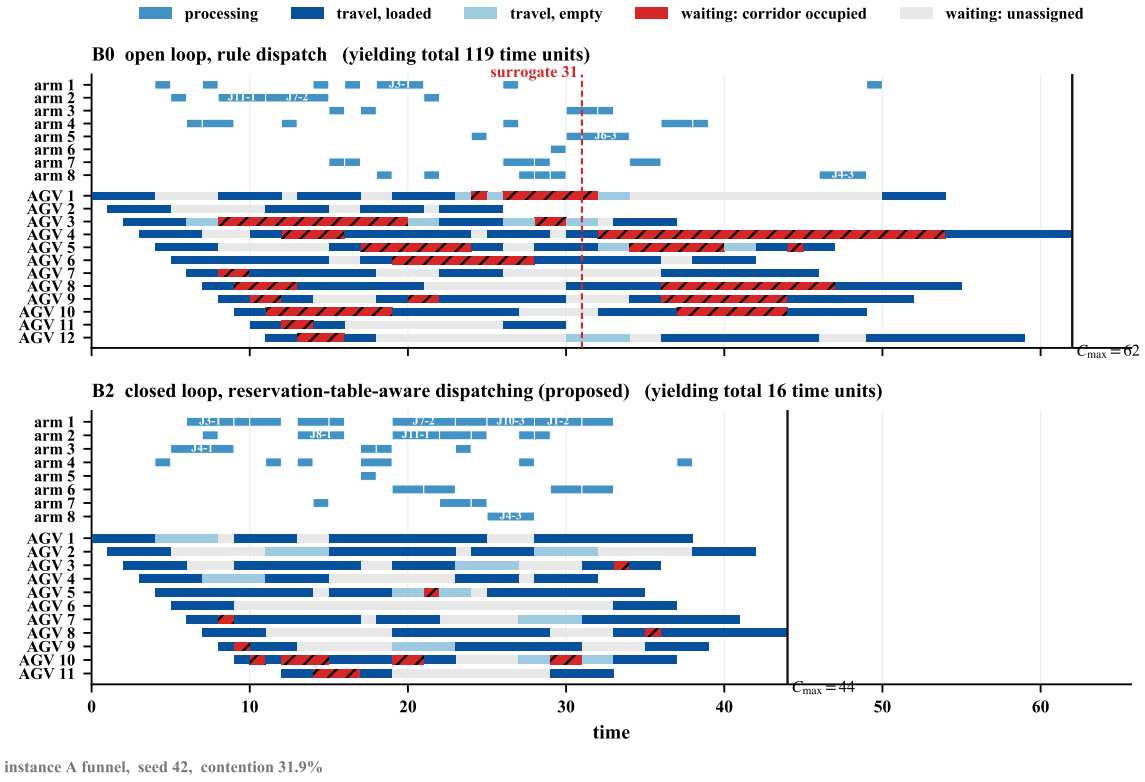


Fig. 8. B0 (top) and B2 (bottom) on the same funnel instance. Robotic arms and vehicles share the time axis; vehicle rows distinguish empty and loaded legs, as well as two kinds of gap. A gap between the two legs of the same transport task: the vehicle waits at a berth because a corridor is occupied, i.e. a yielding wait, which is the delay treated in this paper. A gap between different tasks: the vehicle is idle. The two are shown in different colours so that idle time is not misread as yielding.

5.7 Case study: Gantt chart and critical chain on the funnel instance

Aggregate numbers do not show what happens inside one solve. This section places B0 and B2 side by side on a single funnel instance and colours the links of the critical chain according to Table 3.

Figure 8 draws the robotic arms and vehicles on a shared time axis; vehicle rows distinguish empty and loaded legs, and yielding waits are coloured separately. Three observations follow. In the open-loop solution, yielding waits appear in stretches on the corridors near the load/unload point and concentrate on a few vehicles. The closed-loop solution greatly reduces these waits, and several operations are assigned to an arm that is not the fastest, i.e. assignments of the kind described by Eq. (14), which trade a longer processing time for a shorter path with better traffic conditions. The closed-loop solution also balances vehicle load more evenly; this is a consequence of the first two points rather than a separate mechanism.

Figure 9 colours each link by the five labels of Table 3. On the B0 chain, `corridor` links take a substantial share; on B2 they largely disappear and are replaced by `machine` and `upstream`. This is the dilution described in Section 5.5.3: once a corridor delay is removed, another constraint becomes tight.

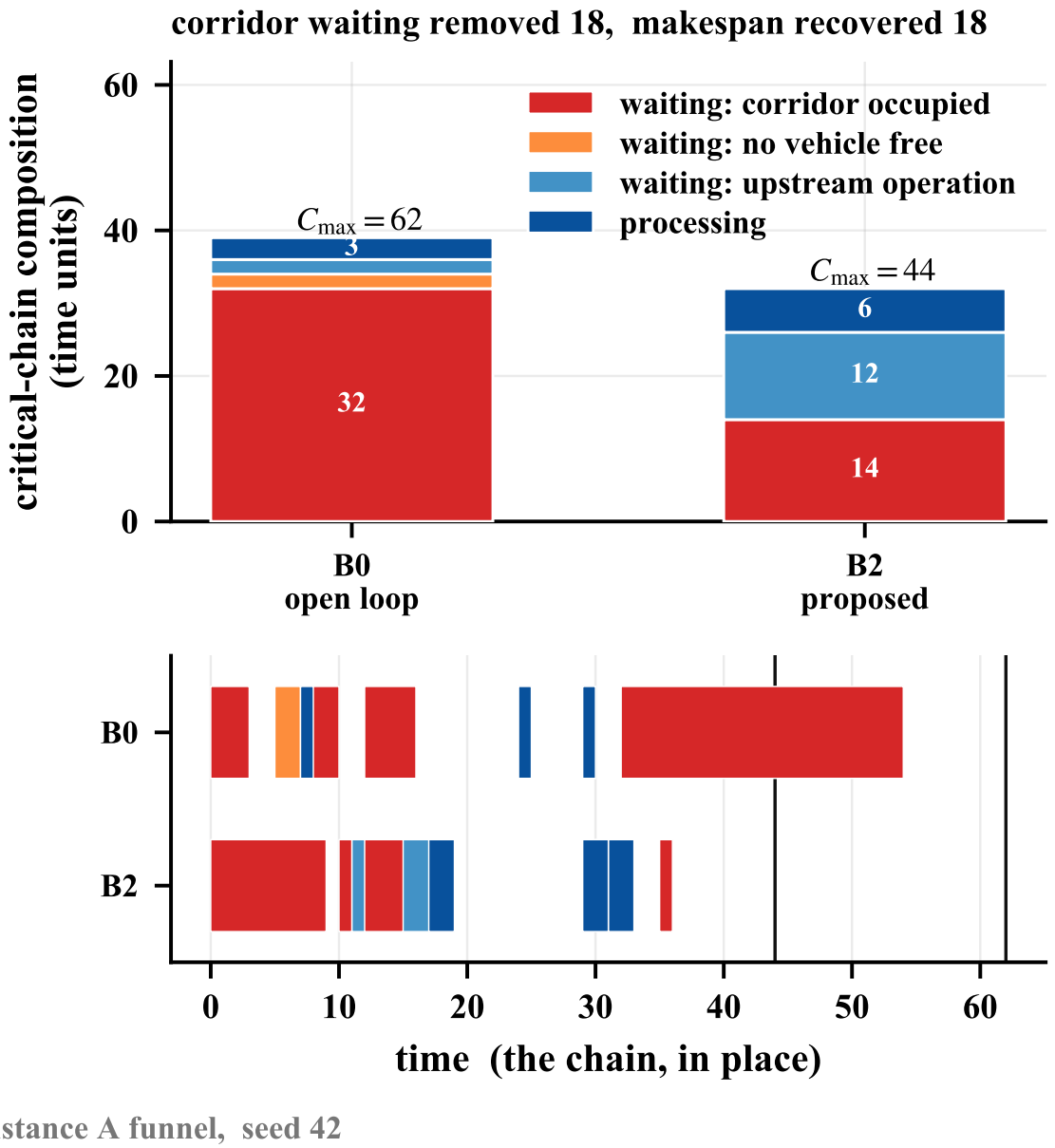


Fig. 9. Links of the critical chain coloured according to Table 3. Top: composition of the B0 and B2 critical chains; the title places "corridor wait removed" beside "makespan actually reduced", and their difference is the part absorbed by the newly tight constraints. Bottom: the same chain positioned on the time axis, showing that the bars above decompose one chain rather than forming a histogram of unrelated events. In the closed-loop solution, `corridor` largely disappears and is replaced by robotic arms and upstream operations.

5.8 Degenerate comparison on public benchmarks

All preceding sections compare different configurations of the same implementation on self-generated instances. This excludes implementation differences among configurations but cannot calibrate the absolute level of the implementation: even if the upper-level search is weak, the differences among configurations remain credible, whereas the absolute makespan reflects only the level of the implementation. This section calibrates the upper-level search on public benchmarks.

What is compared. Once the exclusivity constraint, Eq. (8), is removed, our model degenerates exactly to the standard FJSP_PT (Section 3), which is the *same mathematical problem* as the public benchmarks. We additionally set $\delta_{\text{ret}} = 0$ to match the makespan definition of these instances; this is a model parameter (Table 2), not the removal of a further constraint. After degeneration there is no corridor contention and neither mechanism comes into play, so what is tested is the *upper-level search*, not our framework. The results are reported in a table separate from the self-generated instances, so that the two conclusions do not share one table.

Instances and protocol. We take the 20 instances compiled by Homayouni and Fontes [6] (originating from Fattahi et al. [2]; 2–12 jobs, 4–48 operations, 2–8 machines) and, following the setting of Lim and Moon [11], use two vehicles with no return of finished products. Travel times are taken *as published* from the released matrices, without shortest-path synthesis: Section 5.1 has noted that 2 of these matrices are not the shortest-path closure of any graph, so synthesis would alter the instances. After degeneration the road network G enters the model only through t^*, so the published matrix can be used directly as t^* without requiring a consistent G to exist. Contention requires G itself, which these instances do not provide, so they can be used *only* in the degenerate setting. For two small instances (SFJST01, SFJST03), the published optima were verified digit by digit by hand, confirming that neither includes a return trip. Reference values: 16 instances have a MILP-proven optimum [6], and for the remaining 4 we use the best published values [11].

Results. Table 10 covers 20 instances × 10 seeds. All 200 runs pass three checks: no solution falls below the composite lower bound, no validation fails, and on the 16 instances with a proven optimum (160 runs) no run falls below that value. Because the degenerate setting and the literature pose the same problem, this verifies that the conversion and the conventions are correct. On 12/20 instances the best solution is *identical digit for digit* to the reference value, including all 10 small instances (SFJST01–10); the mean gap over the 20 instances is 2.68% (median 0.00%). The gap rises with size, but not monotonically from one instance to the next: MFJST03 and MFJST06 both do better than their respective smaller neighbours. The four largest instances (32–48 operations) average 9.96%, with a worst case of 12.93%; none of the 16 instances with at most 24 operations exceeds 5.45%.

The gap stems mainly from search capability, not from an insufficient budget. Among the 16 proven optima in the literature, the MILP on the largest instance took nearly ten hours (MFJST06, 34 796 seconds), whereas each of our runs takes only a few seconds. We therefore raised the budget (generation cap five times, early-stopping threshold 3.3 times, measured wall-clock time increased to 2.5 times) and re-ran the 10 MFJST instances in the last ten rows of Table 10—which is where the gaps concentrate. The mean gap of these ten instances falls from 5.37% to 4.53% (the base contains only these ten, a different basis from the 2.68% over the 20 instances above); that is, 2.5 times the computation yields an improvement of only 0.84 percentage points. 5 of them show no decrease, and the instance with the largest gap falls from 12.93% to 12.85%. Under both budgets, *none* of the 300 runs reaches the generation cap; all are terminated

Table 10. Degenerate comparison on public benchmarks (exclusivity constraint removed; the same problem as in the literature). In the "reference" column, OPT denotes a MILP-proven optimum and BK a best published value; the gap is computed from the best solution over 10 seeds. An asterisk marks instances whose best solution is identical digit for digit to the reference value.

			$C_{\max}$ (time units)			
Instance	Ops	Ref.	Value	Best	Mean	Gap
SFJST01	4	OPT	70	70*	70.0	0.00%
SFJST02	4	OPT	111	111*	111.0	0.00%
SFJST03	6	OPT	223	223*	223.0	0.00%
SFJST04	6	OPT	359	359*	359.0	0.00%
SFJST05	6	OPT	123	123*	123.0	0.00%
SFJST06	9	OPT	324	324*	324.0	0.00%
SFJST07	9	OPT	409	409*	409.0	0.00%
SFJST08	9	OPT	269	269*	269.5	0.00%
SFJST09	9	OPT	220	220*	222.0	0.00%
SFJST10	12	OPT	531	531*	531.0	0.00%
MFJST01	15	OPT	485	485*	485.0	0.00%
MFJST02	15	OPT	463	476	483.4	2.81%
MFJST03	18	OPT	482	482*	504.7	0.00%
MFJST04	21	OPT	576	586	623.0	1.74%
MFJST05	21	OPT	532	561	592.8	5.45%
MFJST06	24	OPT	652	677	728.3	3.83%
MFJST07	32	BK	898	965	1023.3	7.46%
MFJST08	36	BK	900	970	1027.0	7.78%
MFJST09	44	BK	1098	1226	1279.2	11.66%
MFJST10	48	BK	1230	1389	1473.0	12.93%

by early stopping, so the larger budget did not lengthen the search. The gap therefore mainly reflects the capability limit of the upper-level search. Narrowing it on large instances is item (3) of Section 6.3.

Sources of the gap between the composite lower bound and the best solution. The median gap between the composite lower bound and the best solution found is 42%. This gap has two components: the amount by which the bound lies below the true optimum, and the amount by which the search solution lies above it. On the self-generated instances the optimum is unknown, so the two components cannot be separated. On the public instances they can. We take only the 16 instances with a proven optimum: the reference of the other 4 is a best published value, the true optimum can only be smaller, and using it as the denominator would overestimate "how far the bound lies below the optimum". On these 16 instances, the same composite lower bound lies a median 10.3% below the proven optimum and a median 10.8% below our best solution; taking the difference instance by instance, the search solution lies a median 0.0 percentage points above the optimum (most instances hit the optimum), and at most 4.1 percentage points. Hence, on these *contention-free* instances, almost all of the gap between the bound and the best solution comes from slack in the bound, not from the search failing to find the optimum. This proportion cannot be transferred directly to the self-generated instances. The composite lower bound is constructed without corridor contention; on the self-generated instances the true optimum can only rise or stay the same because of contention while the bound does not change, so the distance between the bound and the true optimum is no smaller than on the public instances. What can be established is that the 42% on the self-generated instances *cannot* be read as a search gap, because the same bound already lies 10.3% below the optimum without contention; the actual distance between the search and the optimum remains unknown (Limitation (3) in Section 6.2).

6 Conclusion

This paper studies scheduling problems in which flexible assignment and conflict-free AGV routing must be decided together, focusing on shops where transport takes longer than processing and congestion concentrates on a few shared corridors: in such shops, when the corridor to a fast arm is congested, a slower robotic arm with better traffic conditions may be the better assignment. Our bilevel framework lets information from lower-level routing enter decisions through two paths, evaluation and dispatching, and measures the gain and cost of each separately under the same wall-clock budget. The main conclusions are as follows.

6.1 Summary

(1) Instance families with controllable contention intensity. Three families of controlled instances, varying layout, vehicle-to-arm ratio, and flexibility, allow contention intensity to be varied on its own. On them, the closed-loop gain broadly rises with contention intensity, and on the most congested instance it is about two and a half times that on the least congested one, but the relationship is not monotonic (Section 5.4).

(2) A closed-loop bilevel framework with routing-in-the-loop evaluation. The upper level searches assignment and sequencing with a memetic algorithm, and the lower level performs conflict-free routing by time-window reservation; the fitness of each candidate solution is the makespan after the lower level has completed all transports. Proposition 2 and Lemma 1 guarantee that every chromosome is decodable and that the evaluation is consistent with the conflict model of Section 3, with no repair operator needed. Relative to the open loop, the closed loop reduces makespan by 18.86% ($p < 10^{-4}$) under rule-based dispatching and by 17.75% ($p < 10^{-4}$) under reservation-table-aware dispatching.

(3) Reservation-table-aware dispatching and two speed-ups that do not change the output. Before a vehicle is selected, a trial query on the reservation table is run for both transport legs, and the earliest feasible delivery time replaces the unimpeded shortest-path estimate; within the closed loop this reduces makespan by a further 2.45% ($p = 0.0054$). The rule raises the cost per evaluation to 12.2 times that of rule-based dispatching, enough to offset its gain under the same wall-clock budget. Admissible lower-bound pruning and winner-path reuse change neither the selected vehicle nor the timetable (Proposition 3, also confirmed by a bitwise self-check on six instances); they cut router calls by 47.4% and lower the cost multiple to 5.23.

(4) The two paths differ in magnitude, each is effective, and neither can substitute for the other. The closed loop remains significant after Holm correction on 9 of the 10 instances; reservation-table-aware dispatching is significant under both the open and the closed loop only when the instances are pooled for testing (4.59%, $p < 10^{-4}$; 2.45%, $p = 0.0054$, 95% interval [0.94%, 5.19%]). The end-to-end gain 21.35% is slightly below the prediction under independent composition, 22.6%; the two mechanisms do not amplify each other. The "schedule first, then resolve conflicts" route, even with reservation-table-aware dispatching, obtains only 4.59%; because its first-stage search does not count yielding, it forgoes the 17.75% reduction from closed-loop evaluation (B0+ → B2).

6.2 Limitations

(1) Scope of the instance families, and why self-generated instances are used. The main experiment fixes the size and varies only the structure (Section 3.7), so we draw no conclusion about how the mechanisms vary with size. The heterogeneity H is fixed at 0.3 in the main experiment. Its effect is tested only on the independent instances of Section 5.4, which are smaller than the main instances and span four H levels; there the gain does not rise with H.

Whether this holds at the main-experiment scale or larger remains untested. The five layout instances vary more than capacity: the three spine instances vary only network capacity, whereas the two grid instances also change topology and distance (the longest trip rises from 10.0 to 18.0; Section 5.1). Only differences among the three spine instances can therefore be attributed to capacity; the grid instances also reflect topology and distance. In addition, contention intensity summarises congestion in a single number and does not separate yielding that reassignment can avoid. These two factors may explain why the gain rises only broadly, not strictly monotonically, with contention intensity (Section 5.4). Self-generated instances are used because this direction has no public benchmark with an explicit road network (Section 5.1).

(2) The dispatching gain is significant only when pooled. The gain from reservation-table-aware dispatching is only a few percentage points, of the same order as the dispersion across seeds, and is significant only on the 100 pooled pairs (Section 5.4); the class of instances on which it appears stably cannot yet be determined.

(3) With corridor contention, the distance of solutions from the optimum is unknown. The upper-level search has been calibrated on contention-free public benchmarks (Section 5.8), but no proven optimum exists for instances with corridor contention. All configurations share the same search, so this does not affect the differences among them, but effects of a few percentage points (such as the 2.45% from dispatching) may be of the same order as this distance.

(4) The surcharge and yield-record-guided local search did not further reduce makespan; if their evaluation cost is reduced on its own, they must be remeasured. Both were assessed in the same implementation under the same wall-clock budget, so what was measured is the trade-off between cost per evaluation and number of evaluations: when all configurations are sped up by the same factor, that trade-off and the sign of the conclusion stay unchanged; only reducing the cost of one interface on its own could change the sign, as the dispatching speed-ups of Section 4.5 illustrate. Hence, if neighbourhood evaluation is changed to re-solve only the perturbed part, they must be remeasured (item (4) of Section 6.3).

(5) Dynamic disruptions such as robotic-arm and vehicle failures are not considered. Assumption A7 already excludes robotic-arm and vehicle failures and dynamic job arrivals from the model. The reported results are limited to the static setting in which the job set and processing times are known over the planning horizon and equipment does not fail. Under failures or online arrivals, the value of a mechanism also includes how fast it repairs a schedule, not only the quality of the schedule it constructs; this aspect is not measured here (item (5) of Section 6.3).

6.3 Future work

(1) Remeasure on larger explicit road networks, and separate contention from chain substitutability. This corresponds to Limitation (1). Since the main experiment fixes the size, the first step is to test on larger explicit corridor networks whether the closed-loop and dispatching gains keep the same magnitude. In layout design, "whether congestion can be bypassed by reassignment" and "whether the critical chain has only one dominant bottleneck" should be treated as two independent factors; the current contention intensity is a one-dimensional measure that does not separate congestion bypassable by reassignment, so a new measure, first validated on instances whose answers are known, is needed.

(2) Raise the number of seeds to determine the class of instances on which the dispatching gain appears stably. This corresponds to Limitation (2). Raise the number of seeds by about an order of magnitude, sample the vehicle-to-arm ratio at denser levels, and test the fleet-size upper bound predicted by the cost law of Section 5.6.

(3) Measure the distance of solutions from the optimum on instances with corridor contention. This corresponds to Limitation (3). Small instances can be solved exactly with the logic-based Benders decomposition of van Os and Basten [21]; on large instances the gap is insensitive to longer search time, so it is the encoding or the neighbourhood that needs improvement.

(4) Reduce cost by incremental evaluation, then remeasure yield-record-guided local search. This corresponds to Limitation (4). The decoder state advances in one direction, so when a perturbation acts only on the sequence after some position, earlier events and reservations are unchanged. Re-solving only the perturbed part amounts to reducing the cost per evaluation of this kind of local search on its own, after which it can be remeasured under the same wall-clock budget.

(5) Measure the speed of schedule repair under robotic-arm and vehicle failures. This corresponds to Limitation (5). Work on the dynamic flexible job shop has already brought machine failures, rush orders, and job arrivals into rescheduling [1, 5, 25]. In that case the value of a mechanism also includes the speed of repairing a schedule, not only the time and quality of constructing one. Whether an evaluator that re-solves from scratch or a method that modifies only a local neighbourhood holds the advantage cannot be carried over directly from conclusions in the static setting.

References

[1] Stéphane Dauzère-Pérès, Junwen Ding, Liji Shen, and Karim Tamssaouet. 2024. The Flexible Job Shop Scheduling Problem: A Review. *European Journal of Operational Research* 314, 2 (2024), 409–432. doi:10.1016/j.ejor.2023.05.017

[2] Parviz Fattahi, Mohammad Saidi Mehrabad, and Fariborz Jolai. 2007. Mathematical Modeling and Heuristic Approaches to Flexible Job Shop Scheduling Problems. *Journal of Intelligent Manufacturing* 18, 3 (2007), 331–342. doi:10.1007/s10845-007-0026-8

[3] Giuseppe Fragapane, René de Koster, Fabio Sgarbossa, and Jan Ola Strandhagen. 2021. Planning and Control of Autonomous Mobile Robots for Intralogistics: Literature Review and Research Agenda. *European Journal of Operational Research* 294, 2 (2021), 405–426. doi:10.1016/j.ejor.2021.01.019

[4] Jianqi Gao, Yanjie Li, Xinyi Li, Kejian Yan, Ke Lin, and Xinyu Wu. 2023. A Review of Graph-Based Multi-Agent Pathfinding Solvers: From Classical to Beyond Classical. *Knowledge-Based Systems* 283 (2023), 111121. doi:10.1016/j.knosys.2023.111121

[5] Yong Gui, Dunbing Tang, Haihua Zhu, Yi Zhang, and Zequn Zhang. 2023. Dynamic Scheduling for Flexible Job Shop Using a Deep Reinforcement Learning Approach. *Computers & Industrial Engineering* 180 (2023), 109255. doi:10.1016/j.cie.2023.109255

[6] Seyed Mahdi Homayouni and Dalila B. M. M. Fontes. 2021. Production and Transport Scheduling in Flexible Job Shop Manufacturing Systems. *Journal of Global Optimization* 79, 2 (2021), 463–502. doi:10.1007/s10898-021-00992-6

[7] Ahmad Reza Jafarian-Moghaddam, Ali Esmailpoor Sanjabi, and Elham Mortazavi Moghaddam. 2026. Integrated tri-objective scheduling and internal logistics optimization in U-shaped flexible flow shops: a mathematical model and metaheuristic framework. *International Journal of Systems Science: Operations & Logistics* 13, 1 (2026), 2714000. doi:10.1080/23302674.2026.2714000

[8] Kazumi Kasaura, Mai Nishimura, and Ryo Yonetani. 2022. Prioritized Safe Interval Path Planning for Multi-Agent Pathfinding with Continuous Time on 2D Roadmaps. *IEEE Robotics and Automation Letters* 7, 4 (2022), 10494–10501. doi:10.1109/LRA.2022.3187265

[9] Jeonghyeon Kim and Junwoo Kim. 2026. Congestion-Aware Scheduling for Large Fleets of AGVs Using Discrete Event Simulation. *Electronics* 15, 1 (2026), 139. doi:10.3390/electronics15010139

[10] Ruiqi Li, Jianlin Mao, Xing Wu, Wenna Zhou, Chengze Qian, and Haoshuang Du. 2026. A New Framework for Job Shop Integrated Scheduling and Vehicle Path Planning Problem. *Sensors* 26, 2 (2026), 543. doi:10.3390/s26020543

[11] Che Han Lim and Seung Ki Moon. 2023. A Two-Phase Iterative Mathematical Programming-Based Heuristic for a Flexible Job Shop Scheduling Problem with Transportation. *Applied Sciences* 13, 8 (2023), 5215. doi:10.3390/app13085215

[12] Wenhao Lu, Shuai Guo, Tao Song, and Yuwen Li. 2022. Analysis of Multi-AGVs Management System and Key Issues: A Review. *Computer Modeling in Engineering & Sciences* 131, 3 (2022), 1197–1227. doi:10.32604/cmes.2022.019770

[13] Xiangfei Lyu, Yuchuan Song, Changzheng He, Qi Lei, and Weifei Guo. 2019. Approach to Integrated Scheduling Problems Considering Optimal Number of Automated Guided Vehicles and Conflict-Free Routing in Flexible Manufacturing Systems. *IEEE Access* 7 (2019), 74909–74924. doi:10.1109/ACCESS.2019.2919109

[14] Leilei Meng, Weiyao Cheng, Chaoyong Zhang, Kaizhou Gao, Biao Zhang, and Yaping Ren. 2025. Novel CP Models and CP-Assisted Meta-Heuristic Algorithm for Flexible Job Shop Scheduling Benchmark Problem with Multi-AGV. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 55, 11 (2025), 8455–8468. doi:10.1109/TSMC.2025.3604355

[15] Jacqueline Otala, Alden Minard, Golshan Madraki, and Seyedamirabbas Mousavian. 2021. Graph-based modeling in shop scheduling problems: Review and extensions. *Applied Sciences* 11, 11 (2021), 4741. doi:10.3390/app11114741

[16] Haoxiang Qin, Yi Xiang, Yuyan Han, Yuting Wang, Junqing Li, and Quanke Pan. 2026. A Knowledge Region Selection Enhanced Quality-Diversity Algorithm for Real-World Flexible Job Shop Scheduling with Automated Guided Vehicles Transportation. *Engineering Applications of Artificial Intelligence* 164 (2026), 113352. doi:10.1016/j.engappai.2025.113352

[17] Behnam Rahimikelarijani, Hamed Fazlollahtabar, and Sina Nayeri. 2020. Multi-Objective Multi-Load Tandem Autonomous Guided Vehicle for Robust Workload Balance and Material Handling Optimization. *SN Applied Sciences* 2 (2020), 1032. doi:10.1007/s42452-020-3002-y

[18] Yong Jin Suh and Jin Young Choi. 2022. Efficient Fab Facility Layout with Spine Structure Using Genetic Algorithm under Various Material-Handling Considerations. *International Journal of Production Research* 60, 9 (2022), 2816–2829. doi:10.1080/00207543.2021.1904159

[19] Jiachen Sun, Zifeng Xu, Zhenhao Yan, Lilan Liu, and Yixiang Zhang. 2023. An Approach to Integrated Scheduling of Flexible Job-Shop Considering Conflict-Free Routing Problems. *Sensors* 23, 9 (2023), 4526. doi:10.3390/s23094526

[20] Karim Tamssaouet and Stéphane Dauzère-Pérès. 2023. A General Efficient Neighborhood Structure Framework for the Job-Shop and Flexible Job-Shop Scheduling Problems. *European Journal of Operational Research* 311, 2 (2023), 455–471. doi:10.1016/j.ejor.2023.05.018

[21] Roel W. M. van Os and Twan Basten. 2026. An Exact Decomposition-Based Approach to the Conflict-Free-Transportation-Constrained Flexible Job-Shop Scheduling Problem. *Computers & Operations Research* 187 (2026), 107342. doi:10.1016/j.cor.2025.107342

[22] Bin Xin, Sai Lu, Qing Wang, and Fang Deng. 2025. A Review of Flexible Job Shop Scheduling Problems Considering Transportation Vehicles. *Frontiers of Information Technology & Electronic Engineering* 26, 3 (2025), 332–353. doi:10.1631/FITEE.2300795

[23] Fayong Zhang, Rui Li, and Wenyin Gong. 2024. Deep Reinforcement Learning-Based Memetic Algorithm for Energy-Aware Flexible Job Shop Scheduling with Multi-AGV. *Computers & Industrial Engineering* 189 (2024), 109917. doi:10.1016/j.cie.2024.109917

[24] Yulun Zhang, He Jiang, Varun Bhatt, Stefanos Nikolaidis, and Jiaoyang Li. 2024. Guidance Graph Optimization for Lifelong Multi-Agent Path Finding. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*. doi:10.48550/arXiv.2402.01446

[25] Fuqing Zhao and Zongsi Fu. 2026. A deep reinforcement learning algorithm with heterogeneous graph and hierarchical attention mechanism for dynamic flexible job shop scheduling problem. *Engineering Applications of Artificial Intelligence* 183 (2026), 116192. doi:10.1016/j.engappai.2026.116192

Received XX XX 2026; revised XX XX 2026; accepted XX XX 2026